

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО
ITMO University

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
GRADUATION THESIS

Обнаружение визуальных аномалий в промышленных средах с использованием RGB-камеры на борту квадрокоптера / Visual Anomaly Detection in Industrial Environments Using Quadcopter's On-Board RGB Camera

Обучающийся / Student Алшавареб Бассел

Факультет/институт/кластер/ Faculty/Institute/Cluster факультет систем управления и робототехники

Группа/Group R4237c

Направление подготовки/ Subject area 15.04.06 Мехатроника и робототехника

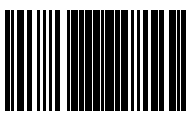
Образовательная программа / Educational program Робототехника и искусственный интеллект / Robotics and artificial intelligence 2022

Язык реализации ОП / Language of the educational program Русский, Английский

Квалификация/ Degree level Магистр

Руководитель ВКР/ Thesis supervisor Колюбин Сергей Алексеевич, доцент, доктор технических наук, Университет ИТМО, факультет систем управления и робототехники, профессор (квалификационная категория "ординарный профессор")

Обучающийся/Student

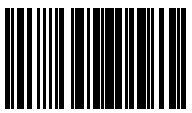
Документ подписан	
Алшавареб Бассел	
17.05.2024	

(эл. подпись/ signature)

Алшавареб
Бассел

(Фамилия И.О./ name
and surname)

Руководитель ВКР/
Thesis supervisor

Документ подписан	
Колюбин Сергей Алексеевич	
16.05.2024	

(эл. подпись/ signature)

Колюбин
Сергей
Алексеевич

(Фамилия И.О./ name
and surname)

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО
ITMO University**

**ЗАДАНИЕ НА ВЫПУСКНУЮ КВАЛИФИКАЦИОННУЮ РАБОТУ /
OBJECTIVES FOR A GRADUATION THESIS**

Обучающийся / Student Алшавареб Бассел

Факультет/институт/кластер/ Faculty/Institute/Cluster факультет систем управления и робототехники

Группа/Group R4237c

Направление подготовки/ Subject area 15.04.06 Мехатроника и робототехника

Образовательная программа / Educational program Робототехника и искусственный интеллект / Robotics and artificial intelligence 2022

Язык реализации ОП / Language of the educational program Русский, Английский

Квалификация/ Degree level Магистр

Тема ВКР/ Thesis topic Обнаружение визуальных аномалий в промышленных средах с использованием RGB-камеры на борту квадрокоптера / Visual Anomaly Detection in Industrial Environments Using Quadcopter's On-Board RGB Camera

Руководитель ВКР/ Thesis supervisor Колюбин Сергей Алексеевич, доцент, доктор технических наук, Университет ИТМО, факультет систем управления и робототехники, профессор (квалификационная категория "ординарный профессор")

Характеристика темы ВКР / Description of thesis subject (topic)

Тема в области фундаментальных исследований / Subject of fundamental research: нет / not

Тема в области прикладных исследований / Subject of applied research: да / yes

Основные вопросы, подлежащие разработке / Key issues to be analyzed

The primary objective of the thesis is to implement and train a defect detector on multiple defect datasets under few-shot setting and deploy it on a quadcopter for performing robotic inspection.

To achieve that, the following tasks must be solved:

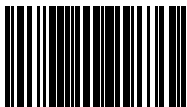
1. Training a few-shot learning model based on transfer learning with an expanding classifier's head, e.g. DKAN on surface defect dataset, and optimizing it based on learning rate.
2. Training another model on oil spill dataset, e.g. YOLO, and optimizing for the learning rate.
3. Verifying the robustness of the models with respect to the conditions of inspection from the quadcopter, in particular brightness, motion blur, and rotations.
4. Training a transformer-based model on both datasets by employing Mixture-Of-Experts.
5. Comparing the results of the MOE model with the results obtained from combining two different models.

Дата выдачи задания / Assignment issued on: 05.03.2024

Срок представления готовой ВКР / Deadline for final edition of the thesis 31.05.2024

СОГЛАСОВАНО / AGREED:

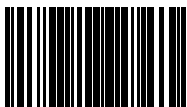
Руководитель ВКР/
Thesis supervisor

Документ подписан	
Колюбин Сергей Алексеевич	
08.05.2024	

(эл. подпись)

Колюбин
Сергей
Алексеевич

Задание принял к
исполнению/ Objectives
assumed BY

Документ подписан	
Алшавареб Бассел	
08.05.2024	

(эл. подпись)

Алшавареб
Бассел

Руководитель ОП/ Head
of educational program

Документ подписан	
Бобцов Алексей Алексеевич	
12.05.2024	

(эл. подпись)

Бобцов Алексей
Алексеевич

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО
ITMO University**

**АННОТАЦИЯ
ВЫПУСКНОЙ КВАЛИФИКАЦИОННОЙ РАБОТЫ
SUMMARY OF A GRADUATION THESIS**

Обучающийся / Student Алшавареб Бассел

Факультет/институт/кластер/ Faculty/Institute/Cluster факультет систем управления и робототехники

Группа/Group R4237c

Направление подготовки/ Subject area 15.04.06 Мехатроника и робототехника

Образовательная программа / Educational program Робототехника и искусственный интеллект / Robotics and artificial intelligence 2022

Язык реализации ОП / Language of the educational program Русский, Английский

Квалификация/ Degree level Магистр

Тема ВКР/ Thesis topic Обнаружение визуальных аномалий в промышленных средах с использованием RGB-камеры на борту квадрокоптера / Visual Anomaly Detection in Industrial Environments Using Quadcopter's On-Board RGB Camera

Руководитель ВКР/ Thesis supervisor Колюбин Сергей Алексеевич, доцент, доктор технических наук, Университет ИТМО, факультет систем управления и робототехники, профессор (квалификационная категория "ординарный профессор")

**ХАРАКТЕРИСТИКА ВЫПУСКНОЙ КВАЛИФИКАЦИОННОЙ РАБОТЫ
DESCRIPTION OF THE GRADUATION THESIS**

Цель исследования / Research goal

The primary objective of the thesis is to implement and train a defect detector on multiple defect datasets under few-shot setting and deploy it on a quadcopter for performing robotic inspection.

Задачи, решаемые в ВКР / Research tasks

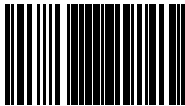
1. Investigating the effect of knowledge distillation during learning novel anomalies. 2. Analyzing and choosing a suitable model with high mean average precision. 3. Analyzing the robustness of the chosen model.

Краткая характеристика полученных результатов / Short summary of results/findings

Knowledge distillation when used in two-phase few-shot learning mitigates catastrophic forgetting. Faster RCNN with the Swin-t transformer as the backbone outperforms all tested models on NEU-DET and oil spills datasets, providing 85% mAP and higher robustness to environmental conditions. The tested models include DINO, YOLOv8, and Faster RCNN with Mixture-of-Experts blended. The model shows high robustness to defects, which have clear, and distinguished features, while it suffers from low robustness to defects, consisting of patterns.

Обучающийся/Student

Документ подписан	
----------------------	--

	
Алшавареб Бассел	
17.05.2024	

(эл. подпись/ signature)

Алшавареб
Бассел

(Фамилия И.О./ name
and surname)

Руководитель ВКР/
Thesis supervisor

Документ подписан	
Колюбин Сергей Алексеевич	
16.05.2024	

(эл. подпись/ signature)

Колюбин
Сергей
Алексеевич

(Фамилия И.О./ name
and surname)

CONTENTS

LIST OF FIGURES	4
LIST OF TABLES	7
INTRODUCTION	10
1 PROBLEM STATEMENT AND LITERATURE REVIEW	12
1.1 Definition and challenges of visual anomaly detection	12
1.2 Faster RCNN	12
1.3 Knowledge Distillation	16
1.4 DKAN	16
1.5 Transformers	19
1.6 Mixture-of-Experts	21
1.6.1 Sparsly-gated Mixture-of-Experts	22
1.6.2 Vision Mixture-of-Experts	23
1.6.3 Reducing Mixture-of-Experts	24
1.6.4 Mixture-of-Experts as domains generalizers	25
1.7 Data cleaning	27
1.8 Metrics used for anomaly object detection.....	28
1.9 Datasets	32
2 SOFTWARE IMPLEMENTATION AND EXPERIMENTAL RESULTS	34
2.1 Software implementation	34
2.2 Experimental results.....	36
2.2.1 Dealing with emergance of novel anomalies	36
2.2.2 Visual analysis of FSCE and DKAN predictions	40
2.3 Performance enhancement with Transformer-based models	43
2.4 Robustness Analysis	47
2.4.1 Modifiers	47
2.4.2 Robustness analysis under few-shot setting.....	48
2.4.3 Analyzing the Swin-based FASTER RCNN	51
2.5 Data cleaning	54

2.6 Discussion	57
BIBLIOGRAPHY	59

LIST OF FIGURES

1	Faster RCNN Architecture.....	13
2	Residual connection	14
3	Feature Pyramid Network.....	14
4	Region Proposal Network	15
5	DKAN architecture.....	17
6	Feature knowledge align loss.....	18
7	Logit knowledge align loss	18
8	ViT transformer architecture	19
9	The attention mechanism	20
10	The Swin transformer	20
11	Shifted-Window Multi-head Attention used in the Swin transformer.....	21
12	Vanilla Mixture-of-Experts.....	22
13	The vision Mixture-of-EXperts	23
14	The Switch transformer	24
15	Knoweldge Integration diagram	26
16	DAMEX dataset division with MOE.....	26
17	Intersection over union	28
18	Non-maximum Suppression	29
19	single-class confusion matrix.....	30
20	predicted and ground truth (Gt) bounding boxes. Predictions are shown in red, while Gts are shown in green.	31
21	Interpolations used for calculating mAP. a) 11-point interpolation. B) All points interpolation.....	32
22	Dependencies used from OpenMMLab.....	34
23	GUI for visualizing outputs and applying modifiers.....	35
24	Pipeline of two-phase learning used in few-shot setting.....	37
25	Optimizing Faster RCNN in the first phase for its learning rate. Each color corresponds to a specific learning rate.	38
26	Confusion matrix of the pretrained Faster RCNN on base classes. .	38

27	Confusion matrix of Faster RCNN after fine-tuning it on novel classes.....	39
28	Comparing mAP of Faster RCNN, FSCE, and DKAN on unmodified data. A) Results on all classes. B) Results on base classes. C) Results on novel classes. Yellow, light blue, and dark blue indicate DKAN, FSCE, and Faster RCNN respectively.	39
29	Predictions of FSCE and DKAN. each subfigure consists of 2 parts, the ground truth at the top, and the predictions at the buttom. The three subfigures above show the predictions of FSCE, while below are the predictions of DKAN.	41
30	Predictions of DKAN. Each subfigure consists of 2 parts, the ground truth at the top, and the predictions at the buttom. The figure shows good predictions as well as incorrect ones.	42
31	Diagram of the ensemble of Swin-t and Swin-s models.	44
32	The Switch transformer inside the Swin transformer in Faster RCNN.	45
33	Switch transformer inside ROI head of Faster RCNN.....	46
34	Correct predictions of Swin-based Faster RCNN.....	47
35	Comparing mAP of Faster RCNN, FSCE, and DKAN on test data with affine transformation applied. A) Results on all classes. B) Results on base classes. C) Results on novel classes. Purple, pink, and green indicate DKAN, FSCE, and Faster RCNN respectively.	49
36	Problems faced with intensive modifiers.	52
37	(a) Original image. b) The image after synthetically adding fog. (c) The image after motion blur is applied.....	53
38	(a) Original image. (b) The image with contrast. c) The image with zoom blur.	53
39	(a) Original image. (b) The image with rotation of 15 degrees. c) The image after rotation of 75 degrees.	53
40	The problem of missing labels.	54
41	(a) The problem of divided labels. (b) The problem of non-rectangular defects.	55
42	Limitations of Swin-based Faster RCNN (a) and (c) misclassification. (b) overlapping defects of different classes.	56

43	samples of predictions of oil	62
44	samples of predictions of crazing	63
45	samples of predictions of inclusion	64
46	samples of predictions of patches	65
47	samples of predictions of pittedsurface	66
48	samples of predictions of rolled-in scale	67
49	samples of predictions of scratches	68

LIST OF TABLES

1	mAP50 of YOLOv8 and swin-based Faster RCNN on NEU-DET and Oil	43
2	mAP50 of all defects of NEU-DET dataset.....	44
3	class-wise mAP of Swin-based Faster RCNN, and the proposed model, when trained on both datasets together.	45
4	class-wise mAP of Swin-based Faster RCNN, and the proposed model, when trained on both datasets together.	46
5	mAP of Faster RCNN, FSCE, and DKAN with and without applying modifiers. The last three columns indicate the results on base classes, novel classes, and all classes together.	49
6	mAP of DKAN on different levels of difficulties of the modifiers....	50
7	mAP of DKAN on different levels of difficulties of the modifiers, after the easy level of modifiers was applied during training.	51
8	class-wise mAP of Swin-based Faster RCNN, and the proposed model, when trained on both datasets together.	55

ABBREVIATIONS

VAD – Visual Anomaly Detection
OD – Object Detection
KD – Knowledge Distillation
FFN – FeedForward Network
ViT – The Vision Transformer
MOE – Mixutre-of-Experts
V-MOE – The Vision Mixture-of-Experts
FLOPs – Floating-point Operations
ROI – Region of Interest.
CNN – Convolutional Neural Network
DNN – Deep Neural Network.
RCNN – Region-based Convolutional Neural Network
IoU – Intersection over Union.
mAP – Mean Average Precision
AP – Average Precision
TN – True Negative.
TP – True Positive.
FN – False Negative.
FP – False Positive.
RGB – Red, Greeb, and Blue.
HSV – Hue, Saturation, and Value

ABSTRACT

The use of quadcopters continues to grow in many sectors, including visual robotic inspection of anomalies. Anomalies are characterized by rare occurrence, and increasing emergence of new types of anomalies. Moreover, the inspection task poses more constraints, implying consideration of the robustness of the solution against various environmental conditions, like light exposure and weather conditions. Deep learning models provide a promising solution for visual anomaly detection (VAD) based on quadcopters. Recent works employ few-shot learning techniques with incremental classifier head under supervised setting. However, they lack the analysis of robustness of their models against environmental conditions. This work aims to provide comparative and robustness analyses of 10 supervised models used in VAD, and utilize Mixture-of-Experts to improve their performance. Experiments were conducted and tested on NEU-DET and oil spills datasets, results show that Faster RCNN with the Swin-t transformer as the backbone outperforms all tested models, providing 85% mAP and higher robustness to environmental conditions.

INTRODUCTION

Industrial inspection is an important part of industrial manufacturing[1]. With the rapid growth of industry, oil consumption is increasing, and the risk of having oil spills accidents escalates[2], which emphasizes the importance of inspection.

The traditional way of performing inspection by humans often raises safety concerns. Quadcopters have proven themselves to be an effective tool to perform this task without threatening our safety, while extending our reachability [2].

One common approach used to perform robotic inspection is by using a camera mounted on quadcopter for running visual anomaly detection (VAD) based on a deep-learning model. However, implementing system for inspection on quadcopter has many challenges. Some of them have to do with the VAD task itself, where data is scarce. In a study on oil spill accidents[3] between 2011 and 2020, the results show that the average number of accidents of oil spills from storage tanks is once in 7.28 years, and accidents of oil spills in the ocean is once in 2.41 years. Due to the low frequency of occurrence of such anomalies, there is not enough data to train the model. This usually leads to overfitting, and it has to be dealt with in a specific few-shot setting in order to avoid it. Several works were done to deal with data scarcity of anomalies. [4; 5]

On the other hand, new types of the anomalies appear from time to time and the model should be able to learn new anomalies with even fewer amounts. This requires a proper splitting of training data in order to maintain data balance.

The dynamic nature of the inspection environment adds another dimension of complexity to the task, which implies having a robust model in order to operate under various environmental conditions. We refer to these conditions by modifiers, and they can range from weather modifiers like light exposure, to quadcopter-related modifiers like motion blur.

The primary objective of this thesis is to implement and train an anomaly model on multiple defect datasets under few-shot settings and ver-

ifying its robustness to different modifiers to deploy it on a quadcopter for performing robotic inspection.

This work begins with a literature review of common models used under supervised learning setting, and an overview of some techniques used to deal with the challenges of VAD, including knowledge distillation, and Mixture-of-Experts. This review will form the background necessary for understanding and analyzing the conducted experiments. Lastly, it provides detailed experiments for analyzing 10 models with and without the absence of modifiers.

1 PROBLEM STATEMENT AND LITERATURE REVIEW

Many challenges are inherent in VAD, part of which are due to the limitations of few-shot learning, and others emerge from issues pertaining to robustness.

This chapter is dedicated to cover the background needed to understand common approaches, which deal with these challenges.

1.1 Definition and challenges of visual anomaly detection

The applications of VAD extend to different industries and encompass diverse types of anomalies, including surface defects, oil spills, etc.

Few-shot object detection is a direction of OD, where the samples are scarce, typically comprising 1-30 samples. This is a common setting in production, since anomalies occur very rarely. Data scarcity can be a big challenge for training a detection model, as it produces unbalanced data, promoting overfitting of the small portion of anomalies in hand, and eventually not detecting new ones. Another issue associated with few-shot learning is the emergence of new type of anomalies. Hence, one of the tasks of the detection model is to detect unseen anomalies. On the other hand, the variant conditions, in which inspection takes place, negatively influences the performance of the detection model in magnitude that varies based on the type of condition and its severity. Common conditions faced by a quadcopter are: brightness and contrast, reflection, motion blur, shadow, etc. Moreover, an anomaly can be located in various angles with respect to the camera, requiring the model to identify rotated objects, which is challenging for CNNs. With the variety of challenges mentioned above, many deep learning techniques must be utilized to mitigate their effects.

1.2 Faster RCNN

The vanilla Faster RCNN [6] is an extension of Fast RCNN, except that it is fully based on deep learning. The architecture of Faster RCNN is shown in Figure 1 It consists of three main parts:

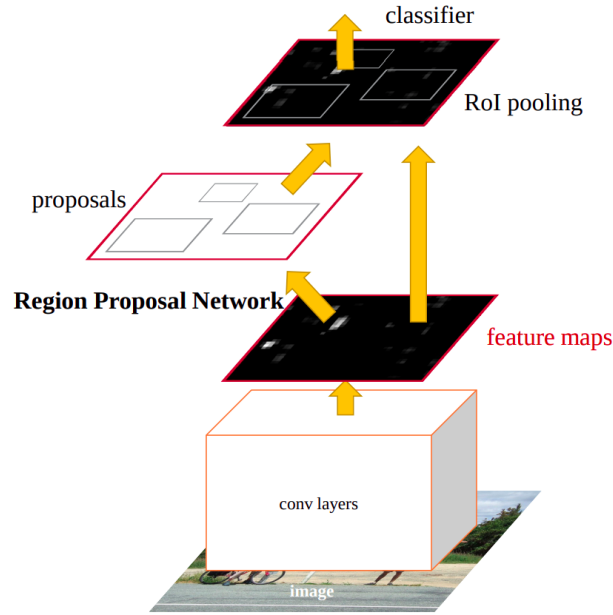


Figure 1 — Faster RCNN Architecture

Backbone

The backbone is a fully-convolutional network which serves as feature extractor. Traditionally, the parameters of the backbone carries the greatest weights within the model. In many applications, a common choice for the backbone for Faster RCNN is Resnet. Resnet is distinguished with its residual connections (also known as skip connections). This kind of connections allow transferring feature maps from previous layers without any processing. If $f(x)$ is a residual function, which is representing a stack of convolutional layers, where x is the input to the stack of layers, then a residual connection allows to estimate the function $f(x) + x$. Using such connections, more semantic information can be obtained by adding up feature maps from different layers in the hierarchy. Figure 2 shows the residual connection. In addition, it allows for having deeper networks without suffering from vanishing or exploding gradient, as there is always a path to the previous layers through the skip connections.

Feature Pyramid Network (FPN)

In FPN, semantic information is collected by retrieving feature maps from different convolutional layers and combining them together. It consists

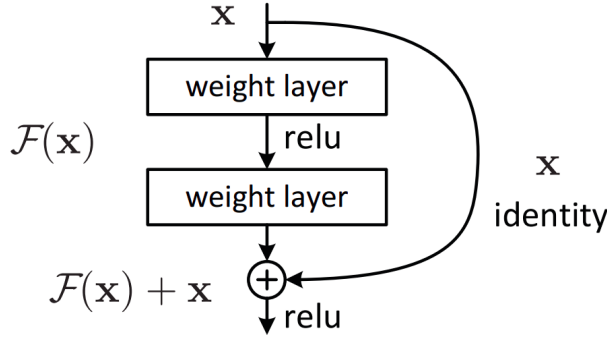


Figure 2 — Residual connection

of two pathways; bottom-up and top-down. In the bottom-up pathway, the feature map is scaled by a step of 2, where the number of channels increase, and the height and width of the feature map decreases. In the top-down pathway, a few upsampling operations are applied, with combining feature map of the same level from the bottom-up pathway, after they undergo 1x1 convolution to make feature maps in the same size. Following this procedure allows merging feature maps with semantic information, but less localization accuracy, with more locationally-precise feature maps, which reinforce the information in the upsampled feature map.

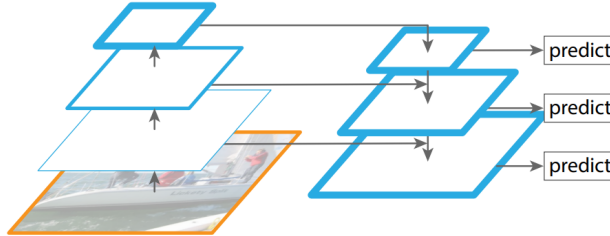


Figure 3 — Feature Pyramid Network

Region Proposal Network (RPN)

Region proposal network is a fully-convolutional network, which identifies regions of interests of the feature map, coming from the backbone. The purpose of using RPN is to define a crop of unique objects from the image and transform the task from multi-object detection to single-object detection. It specifies 3xk anchor boxes, where the number 3 indicates 3 aspect ratios of anchor boxes, and k refers to the number of anchor boxes with k different sizes. Anchor boxes play the role of prior for detections, allowing the model to start from a specific aspect ratio and a specific size, which promotes faster

convergence. More specifically, all detections of RPN are subtracted from the values of anchor boxes, which makes the difference between the current prediction and the ground truth smaller. RPN is class-agnostic, which means it output detections without specifying which classes they belong to. The detections consist of bounding boxes and confidence levels for every bounding boxes for every anchor box. Later, voting is applied to decide which aspect ratio out of the three used to choose for every bounding box. Finally, the detection values are added to their anchor boxes, to convert the bounding boxes back to their size before subtracting.

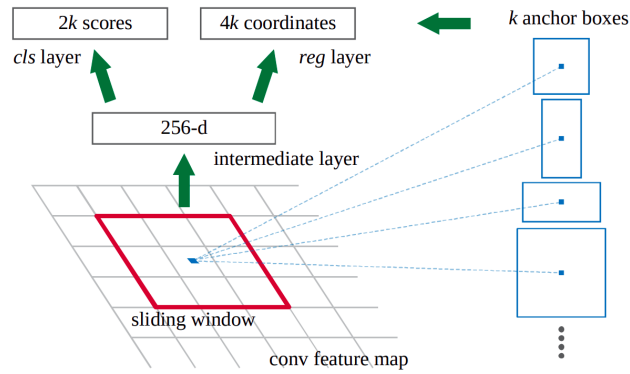


Figure 4 — Region Proposal Network

ROI pooling layer

This layer accepts as an input the feature map, and the bounding boxes indicating regions of interest. With ROI pooling layer, the feature map is cropped into smaller windows and wrapped into the same size, to be fed to the detection head.

Detection Head

The detection head consists of a shared layer, as well as two head; a classifier and a regressor heads. The regressor is responsible for refining the bounding boxes predicted from the RPN layer, while the classifier assigns a label to them. Usually these layers are fully-connected, but they can contain convolutional layers.

1.3 Knowledge Distillation

Knowledge distillation (KD) was first proposed in [7], where softmax with temperature was used to produce a softer distribution of logits, following the teacher-student distillation architecture. This architecture consists of a teacher, which is a cumbersome model trained on abundant training datasets, and having a big number of parameters, and a student, which is a more compact version of the teacher. During training, the student learns to mimic the teacher using a distillation loss. The model ubiquitous application of this training setting is model compression, where the student represents a smaller version of the teacher with the same architecture.

One advantage of using knowledge distillation, as opposed to the typical training setting, lies in training with semantic information instead of 1-hot ground truth. In other scenarios, KD can be useful when we have access to a trained model, but we don't have access to the training data, in this case, knowledge distillation can substitute training data. The amount of distilled knowledge is controlled via the temperature. With a higher temperature applied, the output distribution of the teacher gets closer to the uniform distribution, making differences between variables less noticeable. softmax with temperature is described by Equation 1:

$$p_o = \frac{\exp(\frac{z}{T_o})}{\sum_i \exp(\frac{z}{T_i})} \quad (1)$$

1.4 DKAN

DKAN [8], which is a model based on Faster RCNN, uses knowledge distillation during training to reinforce old states of the model as it is learning. It stands for Dual Knowledge Align Network. It creates two copies of Faster RCNN, one works as a teacher, and the other is the student. The teacher is a trained model with frozen parameters during training, which means that the teacher does not update its weights during training. The input is fed to both the teacher and the student. Then, in addition to the loss coming from the ground truth, DKAN uses two distillation losses coming from the teacher; feature knowledge align loss (FKA), and logit knowledge align loss (LKA). The details of the model are shown in Figure 5 FKA is used to align class-

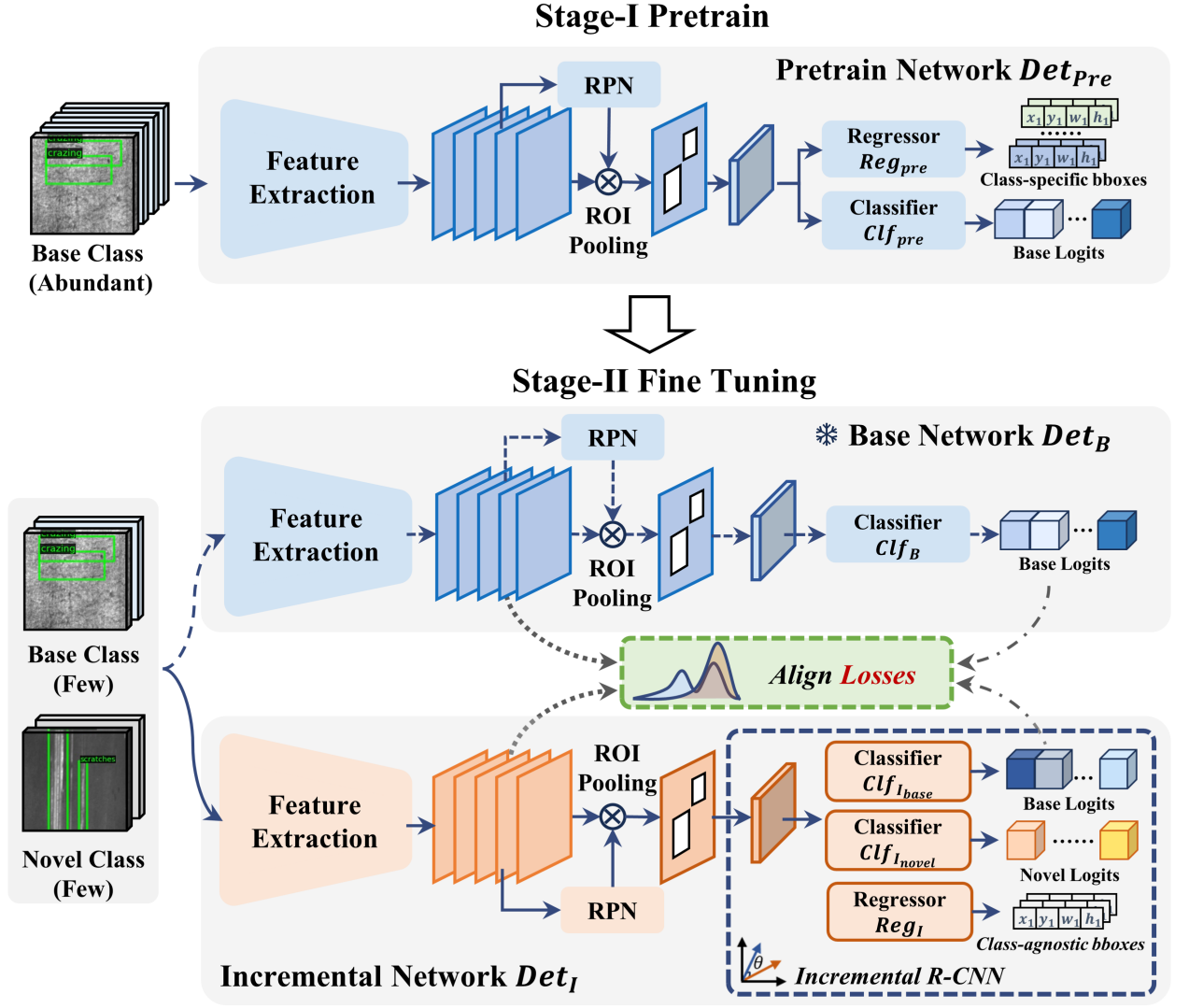


Figure 5 — DKAN architecture

agnostic features between the teacher and the student, while LKA matches the distribution of logits, which are class-specific knowledge, between the models. These losses are shown in Figures 6, and 7.

The overall loss of DKAN is the following:

$$Loss = L_{RPN} + L_{RCNN} + \lambda_1 L_{FKA} + \lambda_2 L_{LKA} \quad (2)$$

where L_{RPN} is the original loss of RPN, and L_{RCNN} is the original loss of the logits of Faster RCNN. λ_1 and λ_2 are the weights of FKA and LKA losses respectively.

In addition to the regression head, the student in DKAN has an expanding classifier that is used to add new classes with further training sessions. To enable effective learning of both old and new classes with different training sessions, cosine similarity classifier is utilized. The cosine similarity classifier

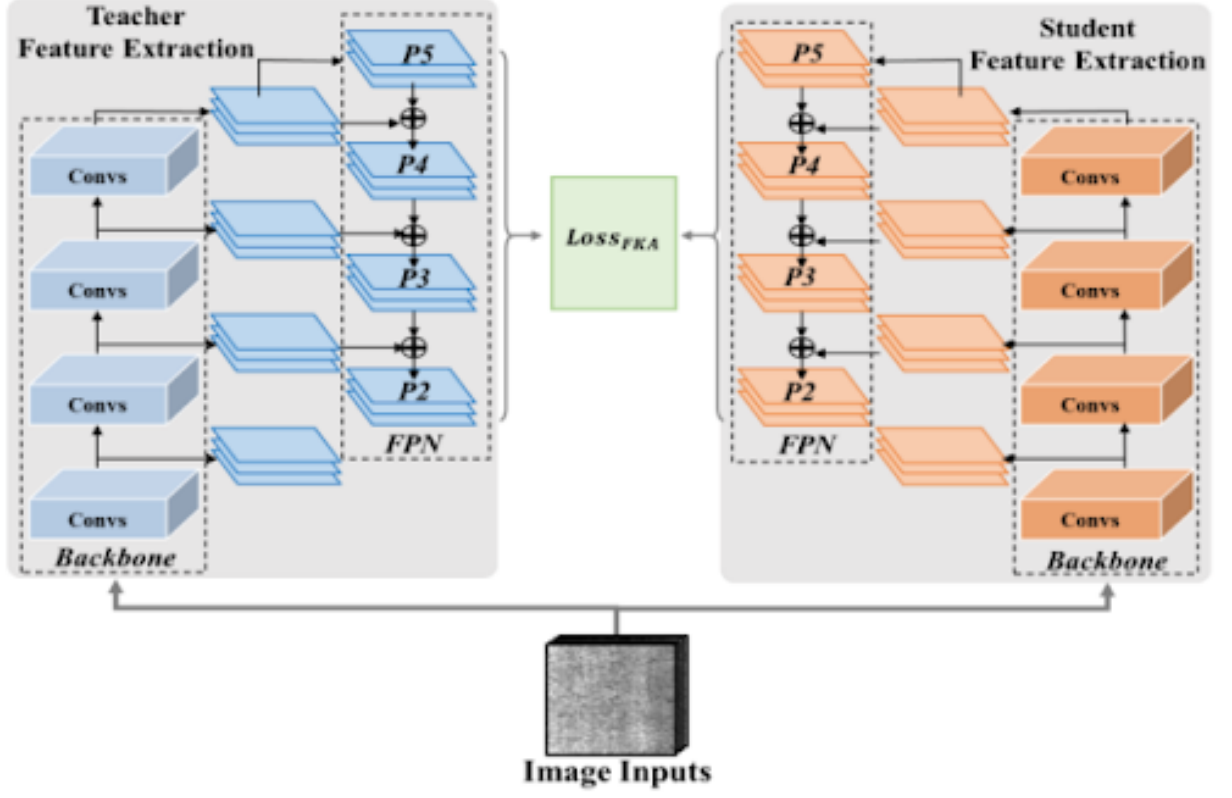


Figure 6 — Feature knowledge align loss

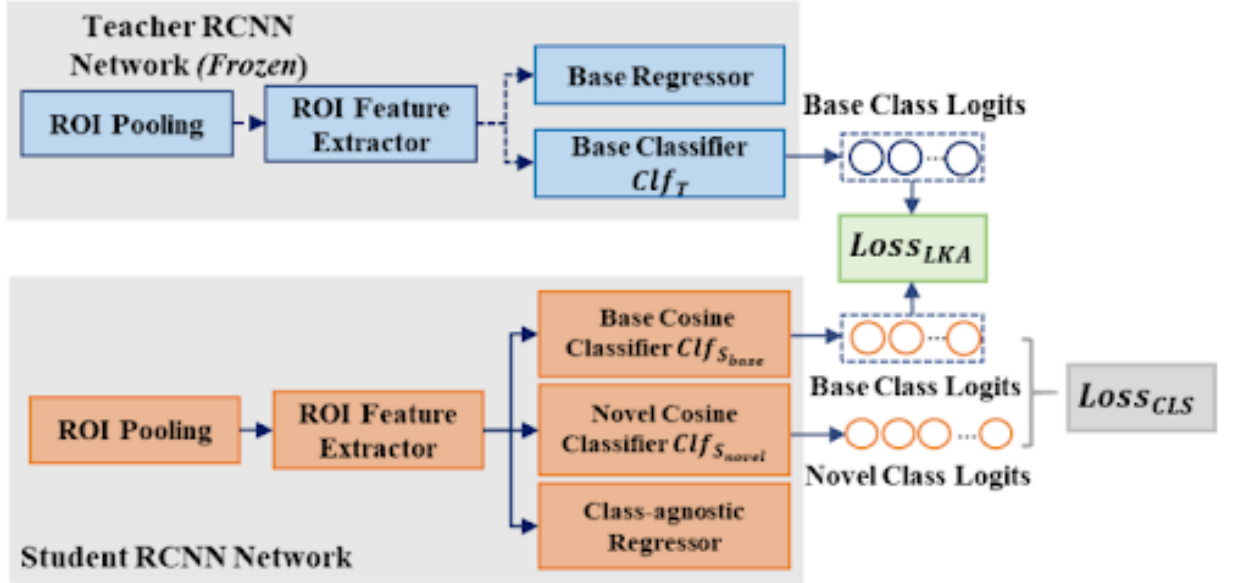


Figure 7 — Logit knowledge align loss

is described in the following equation:

$$c_{i,j} = \alpha \frac{f_i w_j}{|f_i| |w_j|} \quad (3)$$

where α is the scaling factor, f_i is the input, w_j is the weights, $c_{i,j}$ is the cosine similarity between the input i and the class j .

1.5 Transformers

The vision transformer

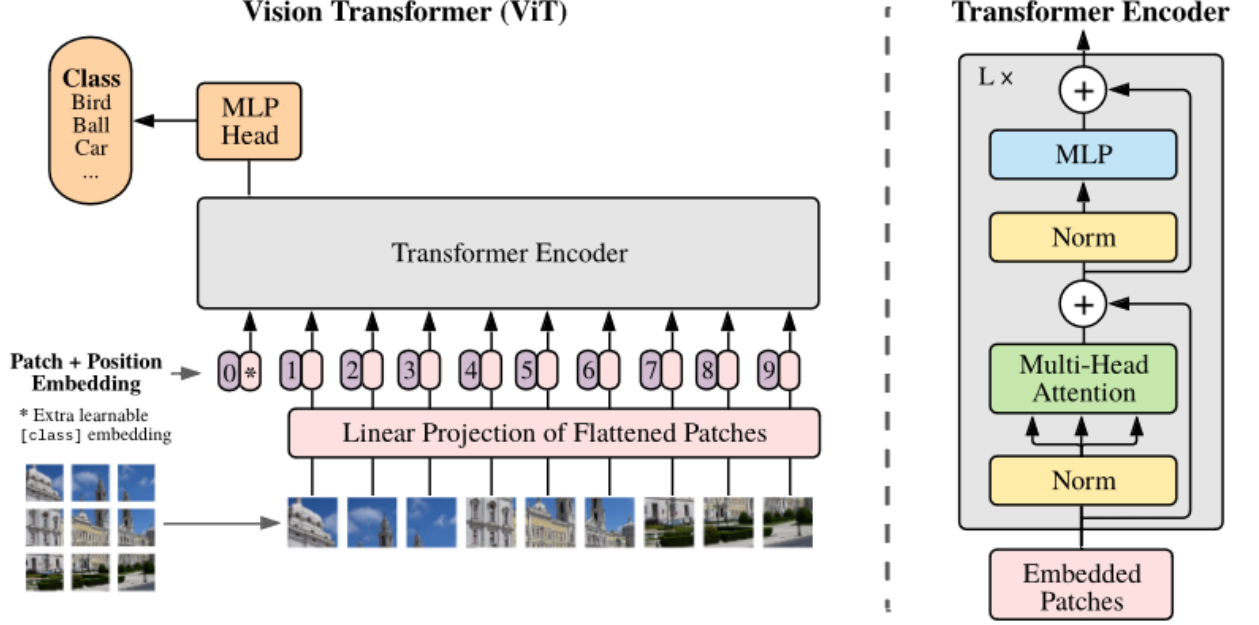


Figure 8 — ViT transformer architecture

Compared with the original architecture of the transformer proposed in [9], ViT utilizes only the encoder part of the transformer. In ViT [10], the image is first divided into $P \times P$ patches, each of which is flattened and mapped into an embedding via a linear projection layer. Then the patches are fed into the encoder after positional embedding is added. Positional embedding is an essential step in ViT, because ViT is permutation invariant, which is an inherent property in the transformer. Applying positional embedding allows remove this property, and at the same time it reserve the position-equivariance property. As a result, the input to the transformer is a concatenation of the flattened patches and position embedding.

The architecture of ViT is shown in Figure 8. Attention matrix is obtained by first calculating the dot product between the keys and queries, then as described in Equation 4, it is multiplied by the values, after it is normalized by $\sqrt{d}$ and softmaxed. The multi-head attention layer consists of h identical attention block, but with different parameters, where the outputs of all blocks are concatenated and fed into a Linear layer. The dimension of the output is usually the same as the dimension of values, keys, and queries

way to reduce computational costs of calculating attention. In ViT, the number of pixels in every patch is 16×16 . It is always preferred to have smaller resolution of patches in order to detect smaller objects, however, this leads to an increment in memory usage, and for big picture it can be impossible to perform the attention operation as the computational cost grows quadratically. The Swin transformer offers a way to decrease the size of the patch, allowing for higher accuracy, without exceeding memory limits. It achieves using the Swin block.

Instead of performing global attention, where every patch is multiplied with all other patches, the Swin block (W-MSA) deals with patches by dividing them into non-overlapping windows, each of them containing $M \times M$ patches. This mechanism reinforces encoding local information, but it fails to calculate attention with respect to other patches outside every window. For that, a shifted-window multi-head attention (SW-MSA), where windows are shifted to cover different patches. The shifted-window mechanism is shown in Figure 11.

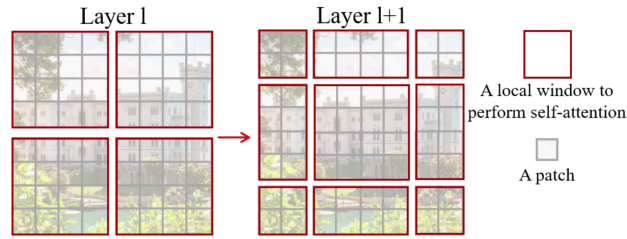


Figure 11 — Shifted-Window Multi-head Attention used in the Swin transformer

Layers using W-MSA alternate with layers using SW-MSA as shown in Figure 10

1.6 Mixture-of-Experts

Using different models for inference on different datasets is not memory friendly, and not preferable in production. Furthermore, training the same model on datasets from different domains is a challenging task, and can lead to wrong predictions. Therefore, generalizing across datasets from different domains is an uprising area of interest in research. This section provides a review of variations of MOE models, and some techniques used in them.

1.6.1 Sparsely-gated Mixture-of-Experts

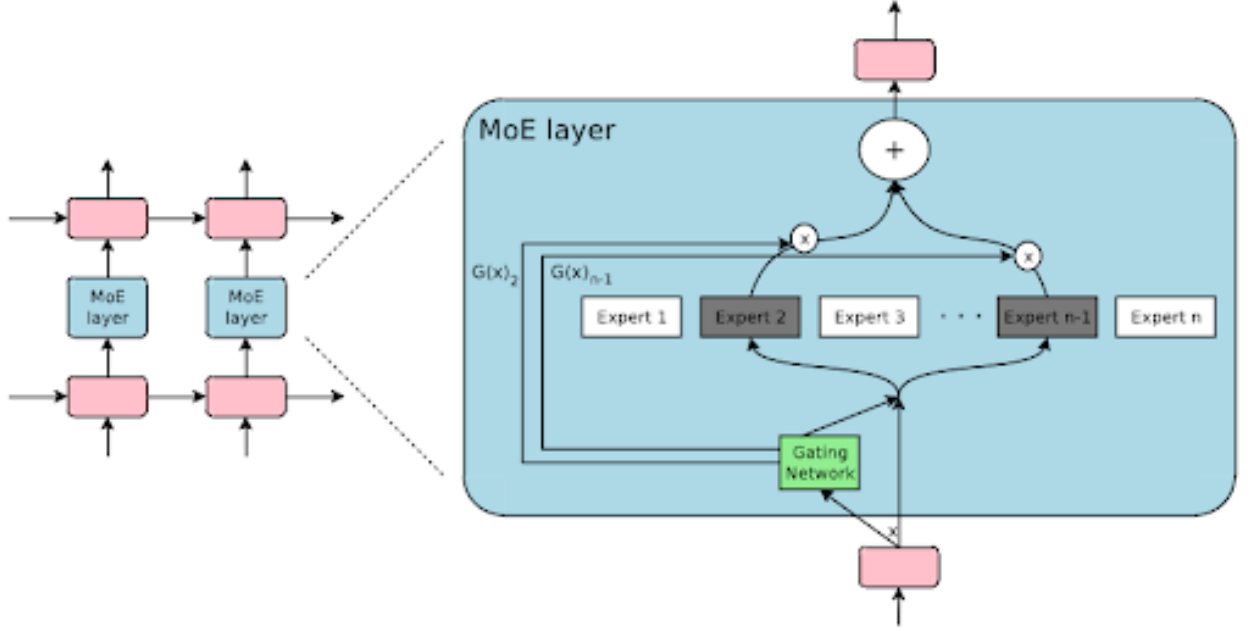


Figure 12 — Vanilla Mixture-of-Experts

In the vanilla sparsely-gated MOE [12], each expert is a dense network placed between stacked LSTM layers, and activated sparsely via a gating network. The gate chooses TopK experts to avoid trivial gradient updates. All experts are trained jointly along with the gating network, making every expert specialized in different subset of the input space. The aim of gate is to learn to map a subset of input dataset into different experts, making experts specialized in different subspaces. One of the main usages of MOE is to scale up the number of parameters without causing the vanishing gradient by utilizing only the needed experts for training samples. Large number of experts is used to allow training on larger datasets. However, in the context of few-shot learning, only small number of experts is needed, since the number of samples is relatively small. The property of sparsely-gated MOEs, which is mostly relevant to this work, is the fast inference time. By utilizing sparsity, only the experts which are responsible for the specific sample will be activated, leading to smaller FLOPS.

Let n denote the number of experts. Then $E_1, \dots, E_n$ are the experts, and G is the gate. The output of MOE for a given input x can be written as

follows:

$$y = \sum_1^n G(x)E_i(x) \quad (5)$$

Where y typically having the same dimensions as x . The gate is a dense layer which produces the probability of choosing every expert. The output is described in the following equation:

$$g_x = W_g x p_i(x) = \frac{\exp(g_{xi})}{\sum_{j=1}^n \exp(g_{xj})} \quad (6)$$

It is important to note that all experts have identical architecture but separate parameters. In most applications found in research [13–15] The experts are FFNs; however, in hierarchical MOE, each expert is itself a MOE.

1.6.2 Vision Mixture-of-Experts

Initially, MOE were used for NLP. The application of MOE in vision was first introduced in [16], and referred to as V-MOE. In V-MOE, MOE

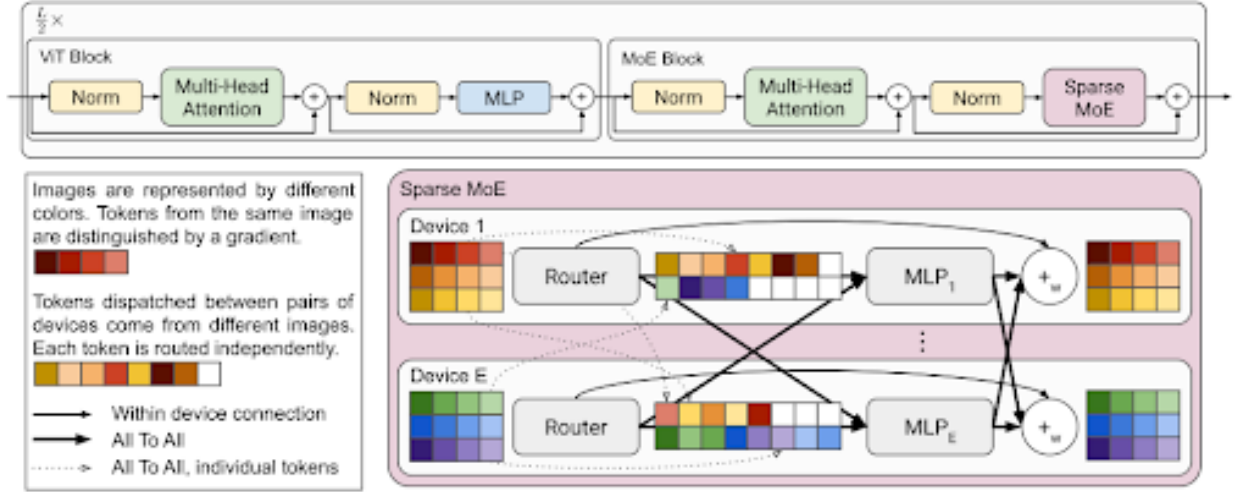


Figure 13 — The vision Mixture-of-EXperts

and ViT blocks are alternating layer wise. The MOE block replaces the MLP layer with an MOE layer. The router used activates the top 2 experts of every image sample. Exploiting the properties of V-MOE allows to train the vision model on a large amount of data.

1.6.3 Reducing Mixture-of-Experts

Encouraging sparsity

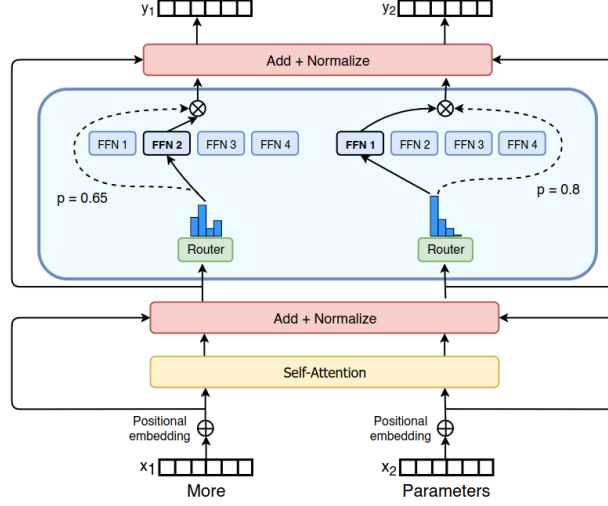


Figure 14 — The Switch transformer

In [12], the authors claimed that in order to get non-trivial gradients, there should be at least 2 experts, of which outputs are combined. In [17], the switch transformer was proposed, in which the router activates only one expert per sample, allowing to reduce computational cost of the router, and obtaining higher performance. In addition, using only one expert instead of k experts allows reducing expert capacity k times. This allowed building models with trillions of parameters, without increasing inference time. To ensure that every expert will have almost the same capacity, a load balancing loss is introduced. In [17] load balancing loss is also referred to as auxiliary loss, and is described in the equation below.

$$Auxiliaryloss = \alpha N \sum_{i=1}^N f_i P_i \quad (7)$$

where P_i is the probabilities of choosing expert i for all samples in the batch β .

$$P_i = \frac{1}{T} \sum_{\beta} p_i(x) \quad (8)$$

p_i is the probabilities of choosing expert i for a sample x in β f_i is the mask for indicating the chosen expert. This term adds a penalty on the chosen

experts, which reduces the frequency of choosing the same expert.

$$f_i = \frac{1}{T} \sum_{\beta} \{\text{argmax}_p(x) = i\} \quad (9)$$

where T is the number of samples in the batch. It is important to notice in the auxiliary loss that P_i is a differentiable function, which the mask f_i is not. N is multiplied by the sum so that the loss will be the same independently of the number of experts. In the case of uniform distribution of the samples over experts, the sum will be $\sum_N f_i P_i = \sum_N \frac{1}{N} \frac{1}{N} = \frac{1}{N}$. Therefore, it is multiplied by N .

Distilling knowledge of Mixture-of-Experts

In [18], they propose knowledge integration task for reducing the MOE structure into a single dense student. Knowledge integrating consists of two subtasks: knowledge gathering and knowledge distillation. Knowledge gathering is used to define the transformation function from experts to a student. The work discusses 4 different knowledge gathering functions: summation, average, top-K, and singular value decomposition. On the other hand, knowledge distillation is used to ensure reducing noise in the distilled knowledge. A diagram of the steps used for knowledge integration is shown in Figure 15. Their idea was inspired by real-world scenario, where a student learns from multiple teachers.

1.6.4 Mixture-of-Experts as domains generalizers

In defect detection, there is a necessity to detect defects, which can come from completely different distributions. In other words, the detector should be able to function on samples drawn from different datasets. One approach is to train a universal model on different datasets. However, it was shown that this causes degradation in the performance, as the updates from different data samples may contradict with each other. On the other hand, a common approach used by practitioners is to concatenate detector heads, where every detector represents a completely different model. Despite the improved performance of this technique compared to the previous one, it is more memory expensive, and it exhibits poor performance in some scenarios.

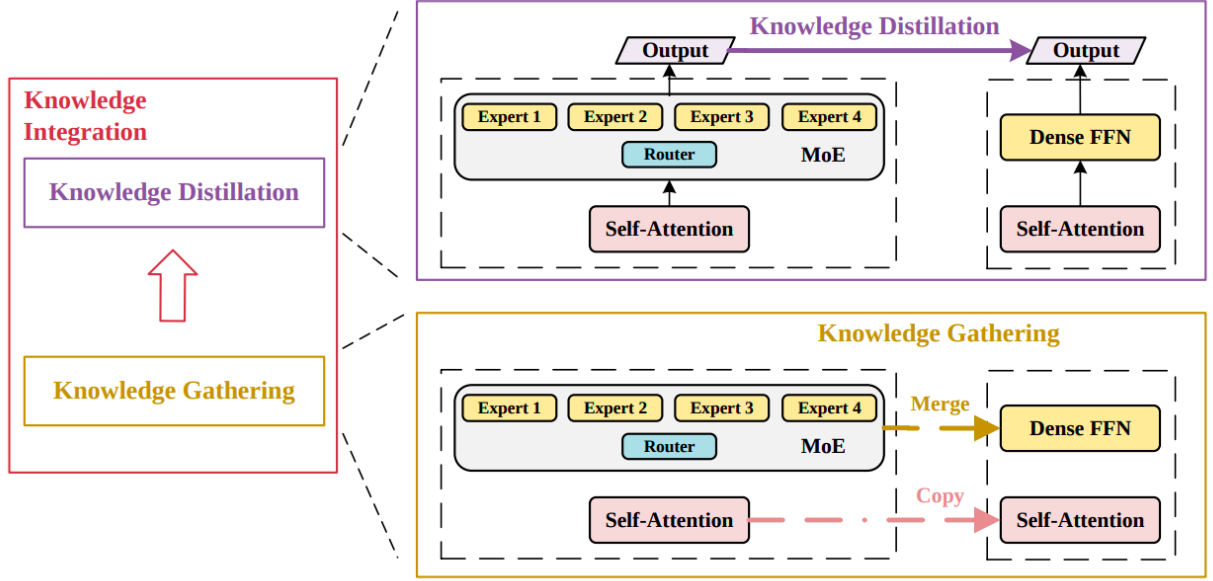


Figure 15 — Knowledge Integration diagram

This can happen when one model shows higher confidence than the others, then it dominates the predictions, even if the prediction is misleading. In [14], a dataset-aware MOE model (DAMEX) is proposed to train separate experts, each on a different dataset, and learn a mapping function to feed samples from every dataset to the corresponding expert as shown in Figure 16.

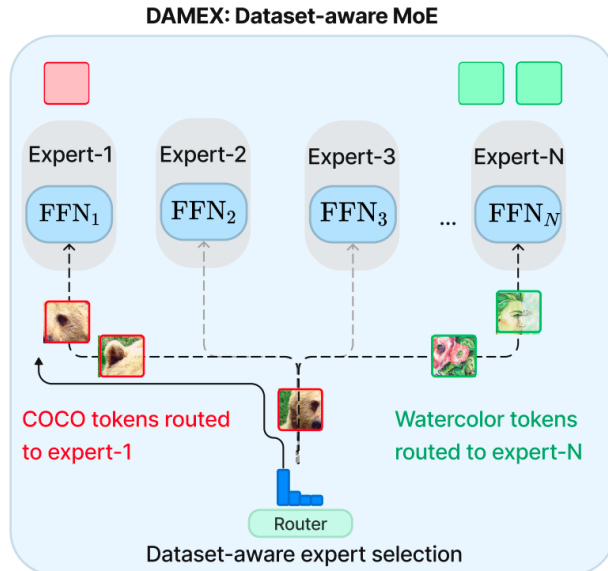


Figure 16 — DAMEX dataset division with MOE

The authors of [14] show that DAMEX handles data imbalance, which is a common problem faced in few-shot learning. This makes DAMEX a promising option for performing visual anomaly detection.

1.7 Data cleaning

Training models on dataset which is not clean, can affect the performance. This effect increases as the dataset gets bigger [19]. Many types of errors can be encountered in annotations of object detection datasets. In this section, three common errors are discussed:

- Overlooked objects, where the object was not seen, therefore it was not labeled.
- Badly located objects, where bounding boxes do not cover the object completely.
- Incorrectly classified objects (is also referred to in literature as swapped objects), where an object is assigned a different class than what it really is.

In the work [20], they introduced a platform for detecting such types of errors. The approach they are using is consists of a pipeline, where the model is trained on the dataset without the image, and then comparing its performance with the model trained on the dataset including the image.

For every bounding box B in image I , the error score is calculated as follows:

$$\sqrt[3]{\prod_{i=1}^3 SoftMin(e_i(I))} \quad (10)$$

where $e_i \in \{e_{overlooked}(I), e_{mislocated}(I), e_{swapped}(I)\}$

To calculate $e_{overlooked}(I)$, for every predicted bounding box $\hat{B}_j(I)$, the minimum confidence score is taken of the bounding boxes, which have no corresponding annotation.

On the contrary, $e_{mislocated}(I)$, $e_{swapped}(I)$ and calculated starting from annotations. To calculate $e_{mislocated}(I)$, annotations are compared only with predicted bounding boxes $\hat{B}_j$ of the same class, S.t. $c(\hat{B}) = c(B_j)$. For every annotation, IOU is calculated for all predictions, which are overlapping with

the annotation. Similarity score is calculated for all overlapping bounding boxes with the annotation, and the Maximum value is assigned.

Finally, the swapped estimate should be considered only when there is high overlap between the prediction and the annotation, with the prediction showing high confidence. In this case, there is a higher chance that there is swapped type error. To calculate $e_{swapped}(I)$, every annotation is compared with all predictions, which are of different classes, and the score is calculated based on the biggest match.

1.8 Metrics used for anomaly object detection

Evaluation metrics can vary depending on the algorithm used in object detection, and the application. A common metric used in object detection is mean average precision (mAP). In this section, we start by introducing basic concepts used in object detection, then we review the method used for calculating mAP.

Intersection Over Union (IOU)

As was stated earlier, the output in the object detection task is bounding boxes, class labels, and a confidence score. Calculating mAP depends on calculating the intersection over union (IOU) between detections of the same class. Let's suppose a ground truth bounding box of a certain label, and

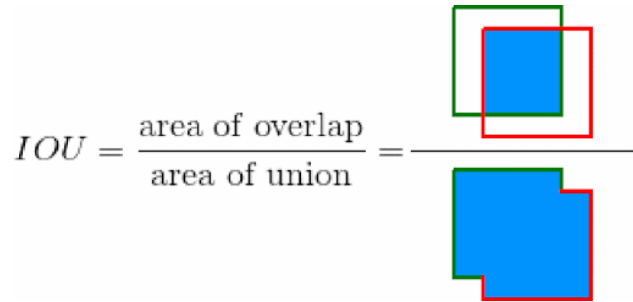


Figure 17 — Intersection over union

a prediction of the same label. And let's suppose that the confidence score is higher than a specific threshold, called confidence threshold, then IOU is calculated as follows:

$$IOU = \frac{Intersection(B_p, B_{gt})}{Union(B_p, B_{gt})} \quad (11)$$

Where B_p is the predicted bounding box, and B_{gt} is the ground truth bounding boxes.

If IOU was bigger than a specific threshold, called IOU threshold, then the prediction is considered TP, otherwise it is considered FP.

It is important to note that all predictions which have lower value than the confidence threshold are removed, and only after that the IOU score is calculated. It is important to note too, that predictions are compared only to Gts of the same class.

Non-Maximum Suppression

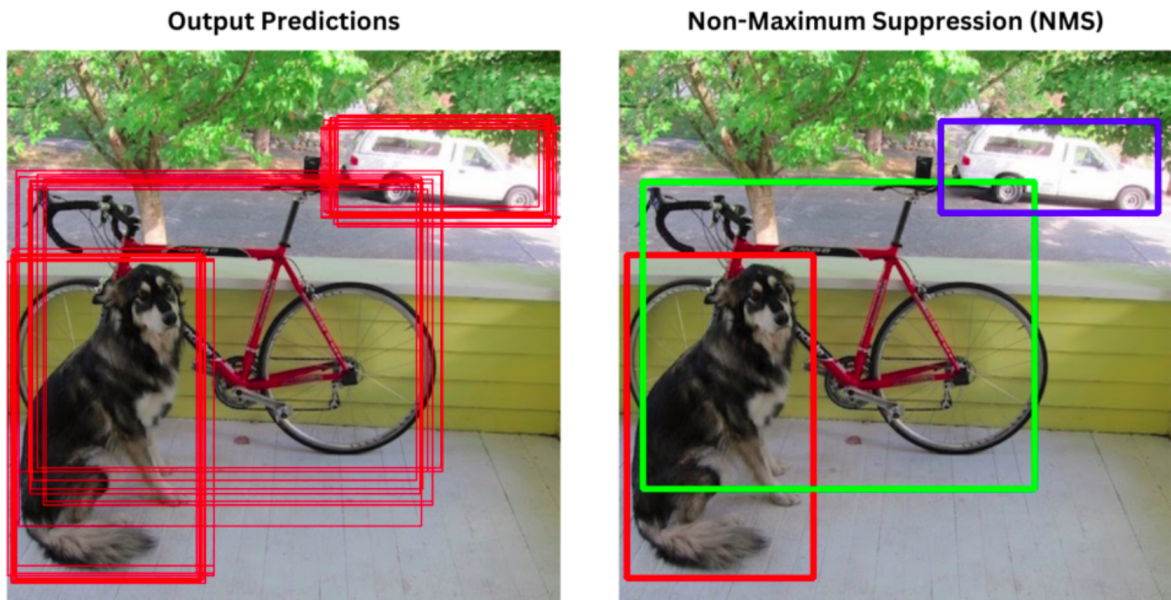


Figure 18 — Non-maximum Suppression

When bounding boxes overlap with each other, there is a decision which must be made to filter some of them, as they show repetitive information. When many bounding boxes point to the same object, the bounding boxes with the highest confidence level is taken, and other boxes which are overlapping with it are removed. However, there are different levels of overlapping between bounding boxes. If the intersection over union (IOU) is higher than a specific value, then the bounding boxes are considered overlapped, otherwise, they are considered pointing to different objects, and are treated separately. It is important to mention that bounding boxes are considered overlapping

only if they are of the same class; overlapping bounding boxes from different classes are treated separately.

Precision and Recall

		Actual Values	
		Positive (1)	Negative (0)
Predicted Values	Positive (1)	TP	FP
	Negative (0)	FN	TN

Figure 19 — single-class confusion matrix

After calculating TPs, and FPs, the precision and recall are calculated as follows:

$$P = \frac{TP}{TP + FP} = \frac{TP}{Allpredictions} \quad (12)$$

$$P = \frac{TP}{TP + FN} = \frac{TP}{AllGroundtruths} \quad (13)$$

Here FN indicates a missing object. In other words, the model did not find the object. In object detection, there is nothing as True Negative, since there is an infinite number of bounding boxes in the background, where there are no objects. Considering that, any metric that is based on TN can not be used for object detection.

Precision indicates how accurate the model is in its predictions. Recall shows how many Gts were detected.

Calculating mAP

Let's consider all predictions of a model and all Gts. To calculate mAP, AP of every class must be calculated first as follows:

1. All bounding boxes with smaller confidence than the confidence threshold are removed

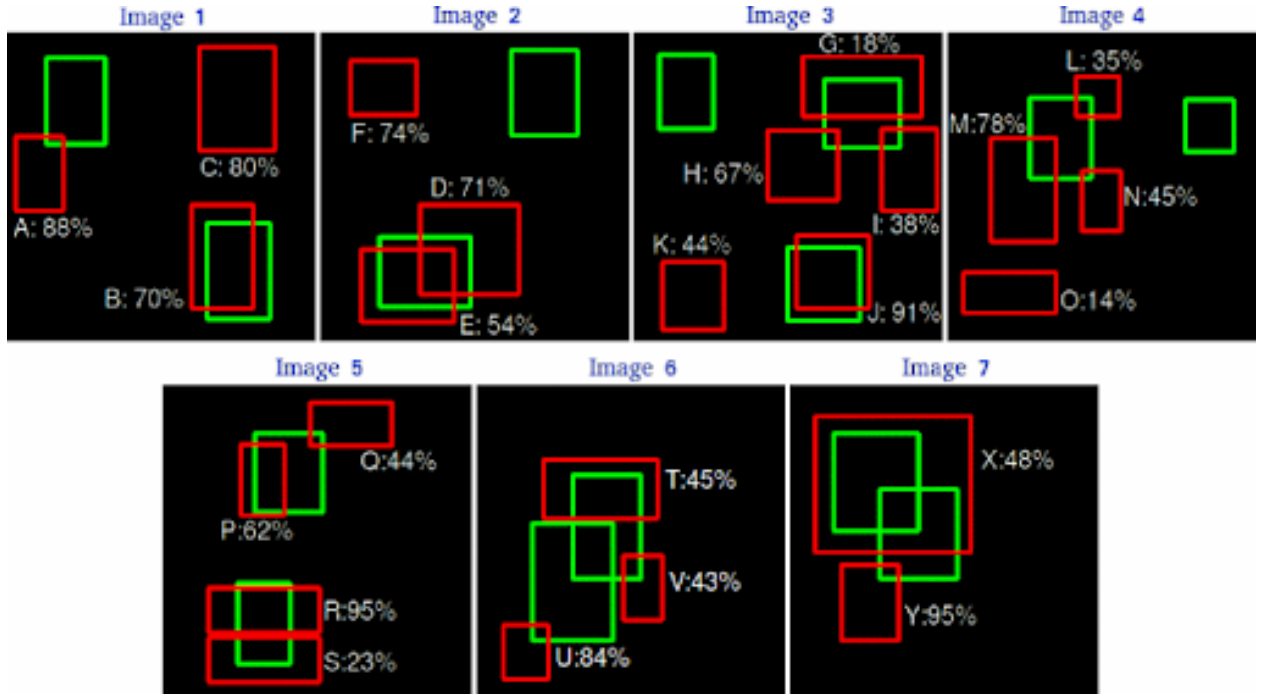


Figure 20 — predicted and ground truth (Gt) bounding boxes. Predictions are shown in red, while Gts are shown in green.

2. predictions as long as Gts are grouped together based on their classes
3. predictions are rearranged in descending order
4. Each prediction is mapped into its corresponding ground truth and IOU is calculated
5. The accumulated TPs and FPs are calculated based on the IOU threshold, moving from predictions with the highest confidence to those with lower confidence.
6. Interpolation method is applied for the precision-recall curve and AP is calculated.

This procedure is repeated for all classes, and then the average of all APs is mAP.

Interpolation methods

There is 2 common interpolation methods used in object detection.

11-point interpolation

In this variation, the curve is quantized into 11 point ranging from 0 to 1 with a step of 0.1. Every point is assigned the maximum value among

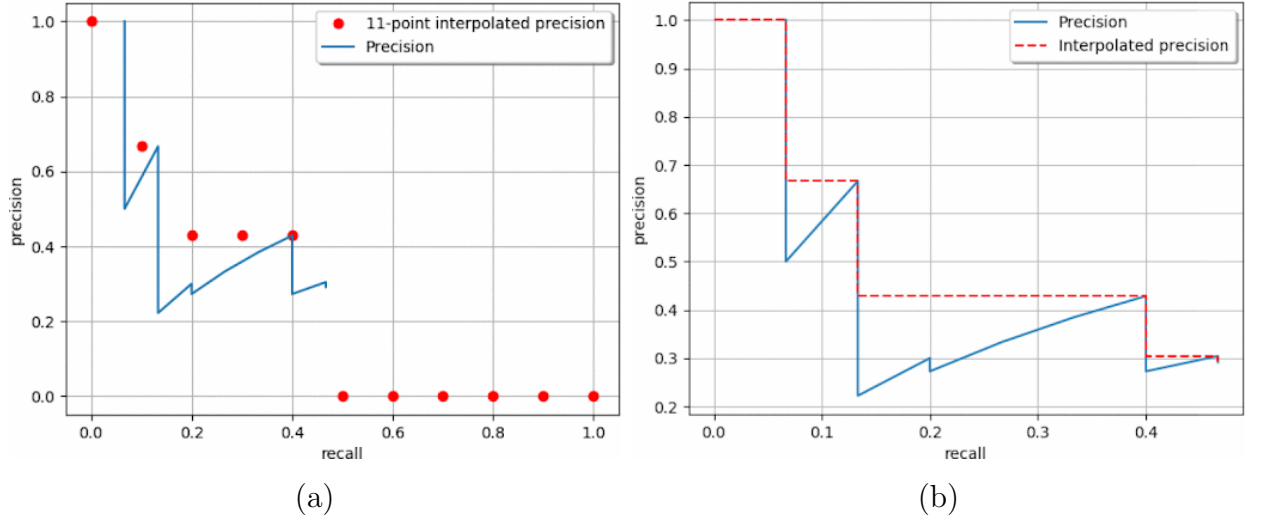


Figure 21 — Interpolations used for calculating mAP. a) 11-point interpolation. B) All points interpolation

other points located to the right of it. Then AP is calculated

$$AP = \frac{1}{11} \sum_{R_i=0}^1 P_{interp}(R_i) \quad (14)$$

where P_{interp} refers to the resulting precision after interpolation.

Interpolation with all points

This method iterates on every point, assigns the maximum value of all points located to the right of it, and then draw lines between points which have the same assigned precision value. Then the area under the curve is calculated as follows:

$$AP = \sum_0 P_{interp}(R_{n+1})[R_{n+1} - R_n] \quad (15)$$

Where P_{interp} refers to the resulting precision after interpolation.

1.9 Datasets

Oil spillage is a worldwide spread problem, which brings disastrous consequences, with its high frequency of occurrence, reaching thousands of spills per year. [21; 22]

On the other hand, surface defects can be critical, which can raise safety concerns. For inspecting surfaces, NEU-DET dataset was used.

Details of the used datasets:

- Oil spillage dataset: it consists of 480 training samples, and 220 test samples. Some images contain fake shadow, which is a useful technique to check overfitting.
- NUE-DET is a dataset for steel surface defects. It contains 6 types of surface defects, with 300 images for every defect. The defects are: crazing, inclusion, patches, pitted-surface, rolled-in scale, scratches. The images of every type are split into 240/60 train/test splits. In total, the training samples are 1440, and the test samples are 360.

2 SOFTWARE IMPLEMENTATION AND EXPERIMENTAL RESULTS

2.1 Software implementation

The code is written in Python 3.8 and is build on OpenMMLab platform. 5 packages were used from OpenMMLab which include:

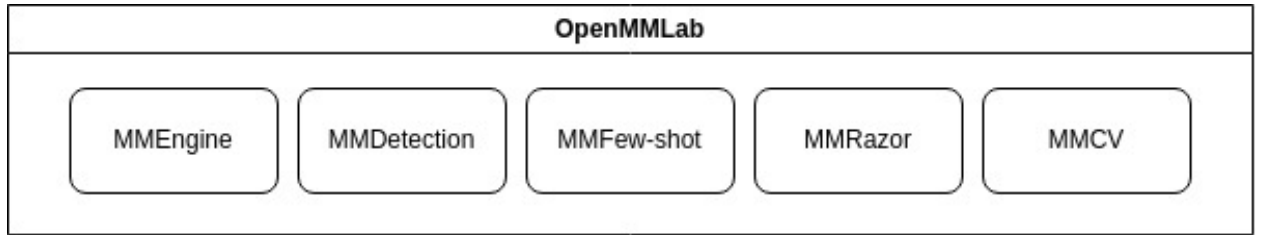


Figure 22 — Dependencies used from OpenMMLab

- MMEEngine for running training experiments
- MMDetection for detection-specific configurations
- MMFew-shot for dealing with data splits which consists of few shots.
- MMRazor for knowledge distillation
- MMCV for image processing

For recording training parameters, ClearML was used. For applying modifiers on datasets, imagecorruptions library was used.

A Graphical User Interface (GUI) was design for visualizing detections and choosing modifiers. The GUI consists of the following components:

- 2 display screens, the left one is for input image, and right one is for the prediction
- Back and Next buttons for choosing different images.
- 3 combo boxes for choosing modifiers. It is possible to apply 3 modifiers at once as maximum
- 3 sliders for controlling the intensity of every modifier.
- Load button to choose a model to load.
- Save button to add the chosen modifiers to the pipeline configuration, and save it in a file for further training and evaluation

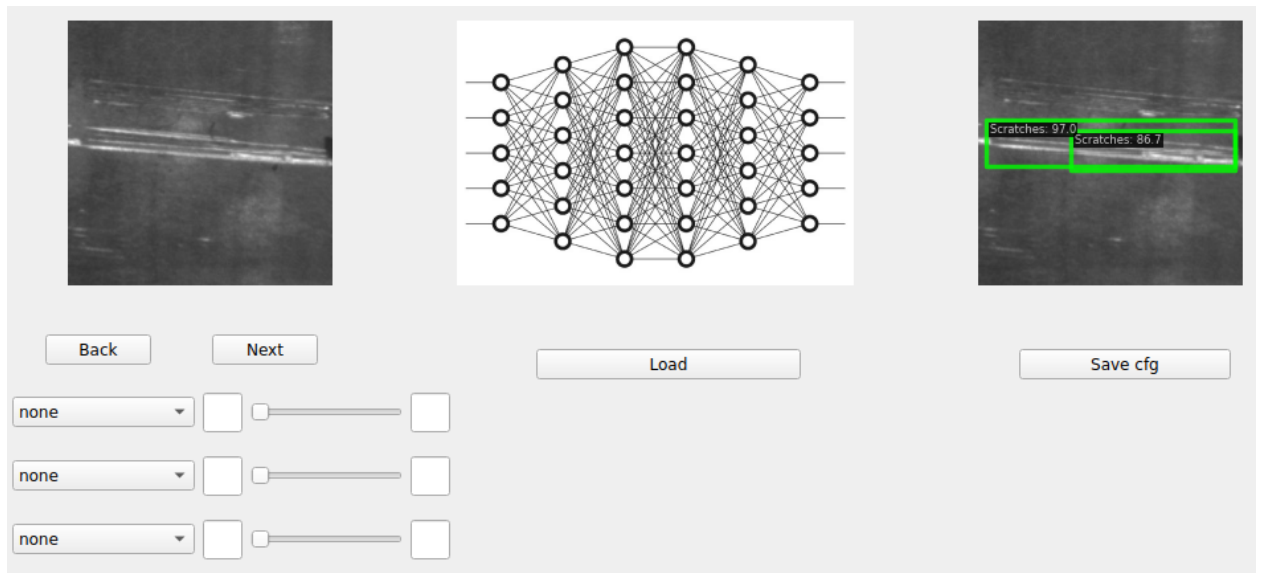


Figure 23 — GUI for visualizing outputs and applying modifiers.

The inference is performed every time a signal is triggered. Signals are triggered by pressing any button, changing the modifier, or moving the slider.

2.2 Experimental results

The main objective of the conducted experiments is to provide extensive analysis of the problems associated with quadcopter inspection, which the detection model suffer from. Consequently, comparative analysis were conducted to select the best choice among the chosen models, which deal with the few-shot problem, the emergence of novel classes, and have high robustness. In the experiments, the performance of different models was evaluated on visual anomalies datasets coming from different domains. In particular, it considers surface defects and oil spillage. The experiments were conducted in the following order: Two models (DKAN and FSCE) were trained and optimized for dealing with problem of the emergence of novel classes. Transformer based models were trained on all data samples without considering the few-shot setting. The model with the best performance was picked for conducting other experiments for improving the performance. MOE techniques were applied to the chosen model as an attempt to improve the performance. The top two performing models were combined as an attempt to improve the performance. Robustness analysis of the chosen models in both the few-shot and the normal settings was conducted. Lastly, data cleaning was applied with the help of the model with the high scores, and the model is re-evaluated for measuring the performance on clean data.

2.2.1 Dealing with emergence of novel anomalies

Few-shot object detection is a direction of object detection, where the samples are scarce, typically comprising 1-30 samples. This section aims to analyze two common object detection models trained under few-shot setting: FSCE[4], and DKAN[5], both of while are based on Faster RCNN.

Learning new emerging anomalies falls under the category of continual learning, which is highly associated with few-shot learning.

Considering that, a two-phase training pipeline (see Figure 24) is used in both FSCE, and DKAN models. At the first phase, the model is trained on a relatively big number of data, called base data. This data encompasses the defect classes available at first. At the second phase, the model is fine-tuned on data of new defects, which is called novel data.

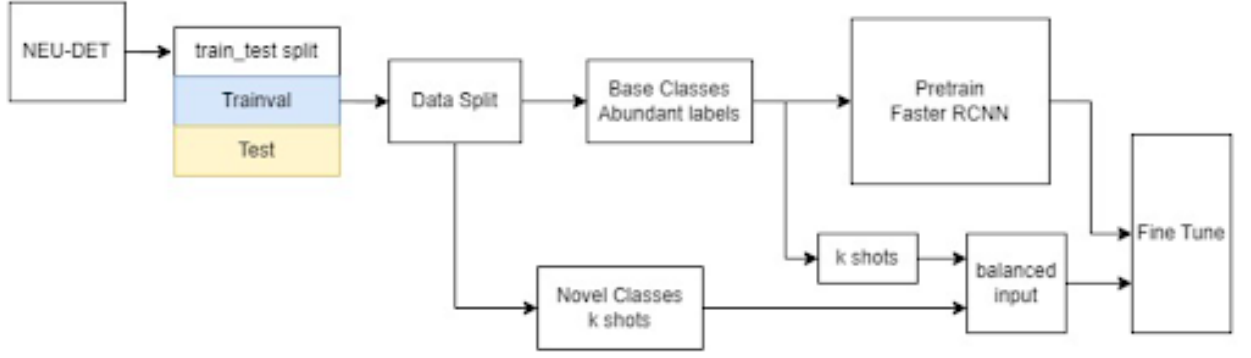


Figure 24 — Pipeline of two-phase learning used in few-shot setting.

One way to fine-tune a model is to re-update its weights on novel classes. However, this training setting promotes overfitting, as it encourages the model to learn novel classes, while forgetting base classes, a phenomenon that is called catastrophic forgetting. This behavior was verified in the conducted experiments. To reduce overfitting at the second phase, K samples (shots) are drawn from every class of base data, such that the model will be trained on equal number of shots of all base and novel classes.

At the second phase, FSCE replaces the classifier’s head with a cosine-similarity head, which is expanded to fit with novel classes. DKAN expands on this method by using knowledge distillation.

To understand the different between retraining Faster RCNN, and training FSCE and DKAN, experiments were conducted by firstly training and optimizing Faster RCNN at the first phase, then training the three models at the second phase.

The dataset used for experiments is NEU-DET, which is an open source dataset for surface defects of the hot-rolled steel strip. The dataset contains 6 types of defects: rolled-in scale, patches, crazing, pitted surface, inclusion, and scratches. The dataset contains 300 samples of every class. In all the experiments, 60 samples are used for testing, and 240 samples were used for training base classes, while only 30 shots were used for training on novel classes.

During the first phase of training, it is important to optimize the performance on Faster RCNN, since its weights will be used as the prior of the model at the second phase. After repeating the training for different learning rates, as shown in Figure 25, a learning rate of 0.01 was chosen, as it showed a mAP of 0.64. Learning rate with higher values than 0.05 caused gradient

explosion. The results of this experiment are shown in Figure 2. This trained model is used for all the following experiments.

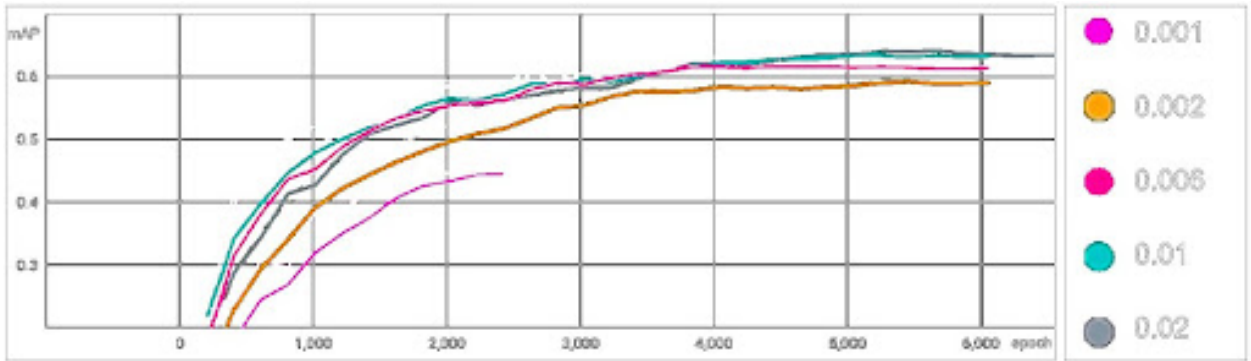


Figure 25 — Optimizing Faster RCNN in the first phase for its learning rate. Each color corresponds to a specific learning rate.

When retraining Faster RCNN on 30 shots of novel classes, the experiment showed a drop in the performance of the model on base classes, which confirms the effect of catastrophic forgetting. The confusion matrix in Figures 26, 27 below shows that model’s accuracy on base classes drops from around 90% to around 65%. As a result, it predicts base classes almost as good as it predicts novel classes. This is because the model is re-optimizing its parameters to fit with novel classes, and doing so, it diverges away from the suitable parameters of base classes.

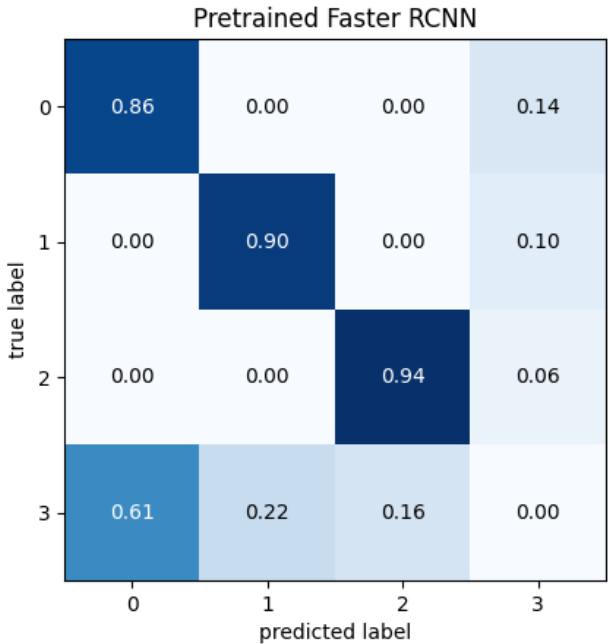


Figure 26 — Confusion matrix of the pretrained Faster RCNN on base classes.

Normalized Confusion Matrix

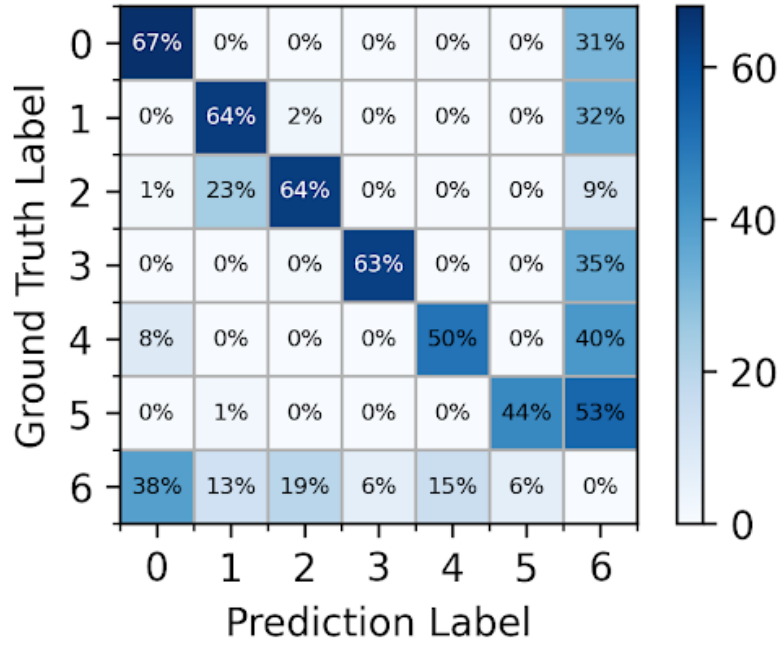


Figure 27 — Confusion matrix of Faster RCNN after fine-tuning it on novel classes.

In the second experiment, DKAN and FSCE were fine-tuned on 30 shots of novel classes. The results are shown in Figure 28. The effect of catastrophic forgetting was observed in the experiments for the three models, where the model’s performance gets worse on base classes as it learns novel classes. The chosen model for performing later robustness analysis is the model with the smallest degradation in performance on base classes, while having acceptable improvement in performance on novel classes. Based on that, DKAN was chosen for performing robustness analysis, see Section 2.4

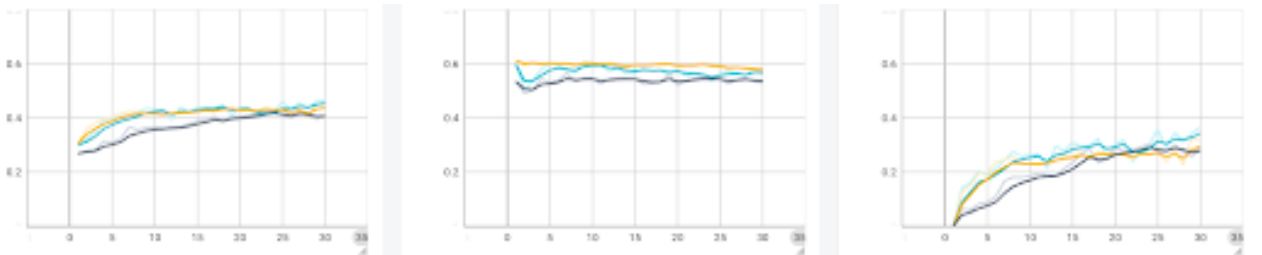


Figure 28 — Comparing mAP of Faster RCNN, FSCE, and DKAN on unmodified data. A) Results on all classes. B) Results on base classes. C) Results on novel classes. Yellow, light blue, and dark blue indicate DKAN, FSCE, and Faster RCNN respectively.

2.2.2 Visual analysis of FSCE and DKAN predictions

The predictions of FSCE and DKAN are shown in Figure 29. As it is demonstrated, DKAN mitigates the misclassification problem, which manifests in FSCE. A possible reason of that is the logit distillation loss used in DKAN, which encourages correct class-specific predictions. However, as a result of mitigating misclassification, it was found that DKAN may miss some defects. This behavior was observed in many samples. Nevertheless, on some samples, DKAN produced misclassified bounding boxes along with correct ones around the same defects. What makes this problem challenging is that non-maximum suppression can not solve it, as it is applied to bounding boxes of the same classes, and applying it on different classes may lead to undesirable predictions. Additional detections of DKAN are shown in Figure 30

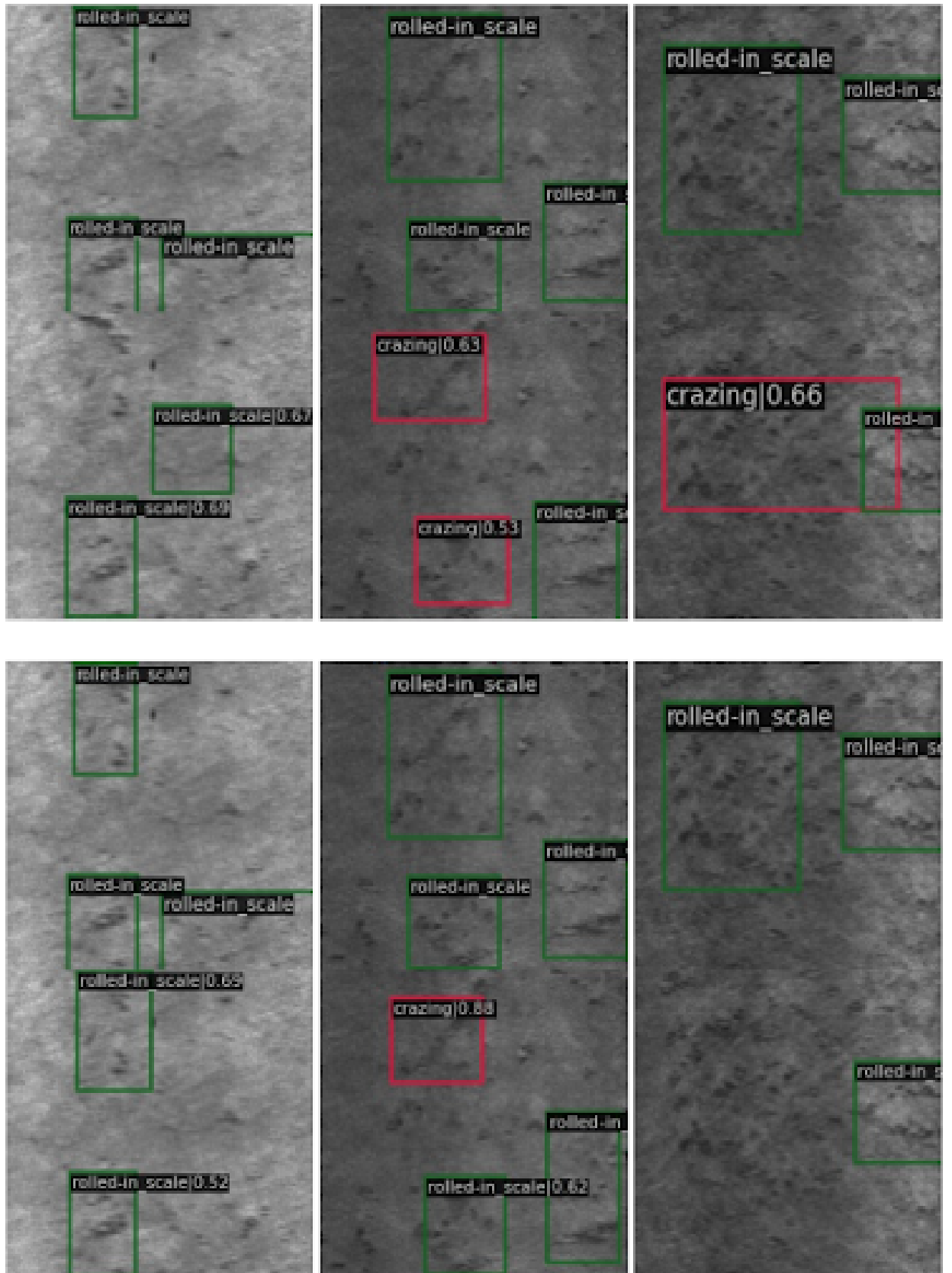


Figure 29 — Predictions of FSCE and DKAN. each subfigure consists of 2 parts, the ground truth at the top, and the predictions at the bottom. The three subfigures above show the predictions of FSCE, while below are the predictions of DKAN.

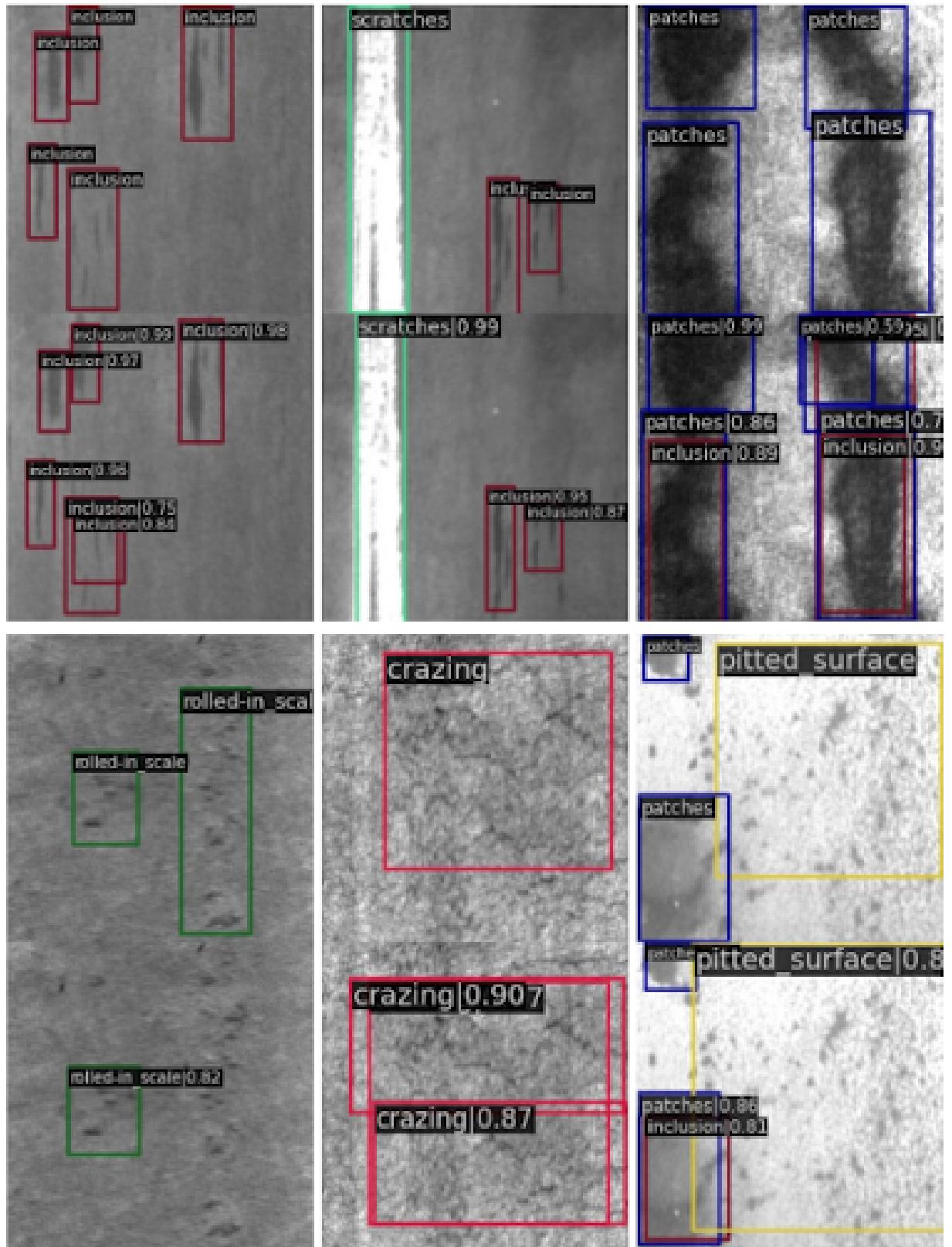


Figure 30 — Predictions of DKAN. Each subfigure consists of 2 parts, the ground truth at the top, and the predictions at the bottom. The figure shows good predictions as well as incorrect ones.

2.3 Performance enhancement with Transformer-based models

Transformer models have shown high potential in NLP and computer vision.

This section contains two sets of experiments that were conducted. In the first part, 3 different transformer-based models were tested on both NEU-DET and oil datasets separately. In the second part, MOE layers were utilized in 2 places of Faster RCNN to observe their performance on NEU-DET datasets, and then on both NEU-DET and oil datasets jointly.

Choosing the transformer model

This experiment aims to select the best model for NEU-DET and oil datasets among the following models:

1. Faster RCNN with Swin transformer backbone.
2. YOLOv8
3. DINO [23]

The experiment was conducted to investigate the performance of these models on both datasets separately. The metric used for evaluation is the mAP50, which is the mean average precision for a IOU threshold of 0.5. The results are shown in Table 1

Table 1 — mAP50 of YOLOv8 and swin-based Faster RCNN on NEU-DET and Oil

Model	NEU-DET	Oil
Faster RCNN	0.7	0.638
YOLOv8	0.55	0.48
DINO	0.58	0.45

Since NEU-DET contains multiple classes, another table is made for having a class-wise evaluation. The results are shown in Table 2 show that Faster RCNN with the swin transformer outperforms DINO and YOLOv8 on both datasets, not only in the mAP, but also in the AP of every class. DINO and YOLOv8 showed opposing performance on datasets, where YOLOv8 scored higher on oil dataset, and DINO scored better on NEU-DET dataset.

Table 2 — mAP50 of all defects of NEU-DET dataset

	crazing	inclusion	patches	pitted surface	rolled in scale	scratches
Faster RCNN	0.38	0.84	0.9	0.77	0.5	0.8
YOLOv8	0.253	0.789	0.855	0.323	0.522	0.58
DINO	0.246	0.672	0.792	0.757	0.417	0.576

Combining Swin models

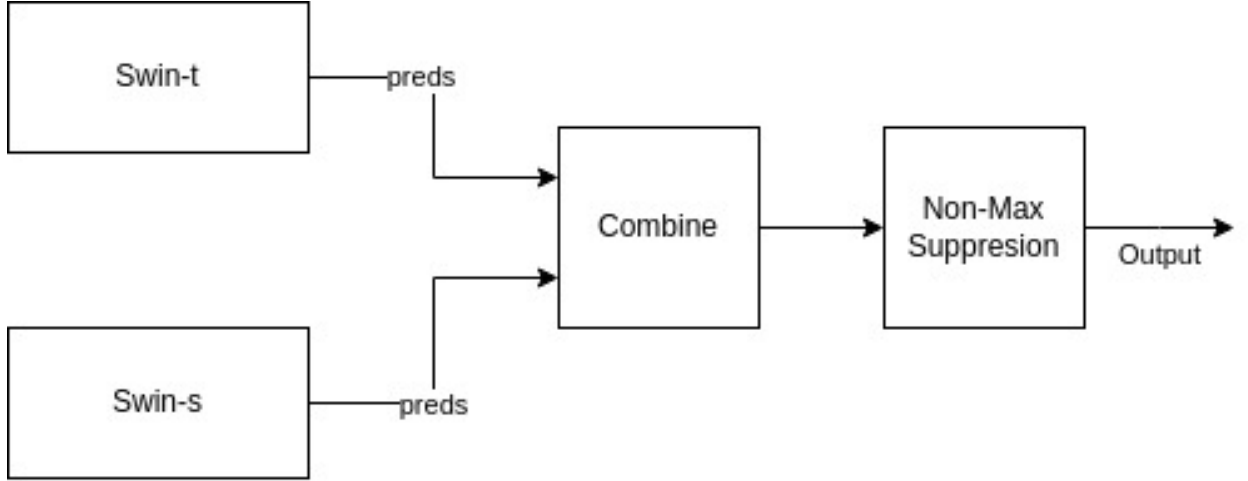


Figure 31 — Diagram of the ensemble of Swin-t and Swin-s models.

With different number of parameters, Swin-t and Swin-s have different recognition capability in different cases. Combining them together can be a good way to use each model’s advantage. The approach used for that, in this work, starts by combining the predictions, then applying non-maximum suppression, where it is chosen from the overlapping detections those with the higher confidence, while non-overlapping detections are added up.

Investigating the performance of adding MOE layer

Using MOE can save a lot of memory, without trading-off inference time. In Faster RCNN, MOE can be incorporated in 4 parts; in the backbone, in the FPN, in the RPN, and in the ROI head. In the conducted experiments, MOE was placed in two positions in Faster RCNN separately. In the first one, MOE was implemented only in the backbone and in the second one it was implemented in the RIO head. The backbone of Faster RCNN used in

the experiments is the Swin transformer, with either the tiny or the small version.

MOE in the backbone

One way to employ MOE in Faster RCNN is to blend it in the backbone. The model architecture used in this part is as follows: Faster RCNN is used with the Swin transformer as its backbone. The FFN in last layer of the Swin transformer is replaced with the Switch transformer. This approach aims to investigate the use of MOE for generalizing among datasets from different domains.

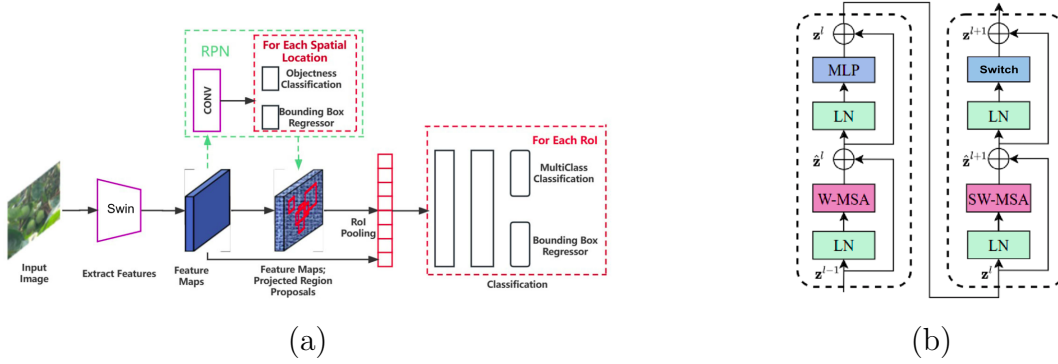


Figure 32 — The Switch transformer inside the Swin transformer in Faster RCNN.

In this experiment, both of the datasets are merged and trained on together. The reason for choosing this setting is to check whether using MOE will improve the ability of the model to generalize across datasets from different domains.

Table 3 — class-wise mAP of Swin-based Faster RCNN, and the proposed model, when trained on both datasets together.

model	crazing	inclusion	patches	pitted surface	rolled in scale	scratches	oil
swin-t	0.33	0.78	0.9	0.74	0.48	0.78	0.62
Switch in Swin-t	0.34	0.77	0.88	0.22	0.39	0.4	0.54

The results shown in Table 3 describe how adding Switch transformer changes the results. As the results show, the MOE swin give similar results to the pure swin for some classes, while the performance on other classes drop significantly. This behavior can be due to overfitting some experts over the others. One way to mitigate this behavior is by modifying the load balancing loss, however, this modification was not considered in the experiments. The

used MOE model did not cause an improvement in the performance compared to the pure Swin model.

MOE in ROI head

In this modification, two shared fully-connected layers of ROI head where replaced with an MOE layer. The MOE layer consists of the switch transformer with each expert representing two consecutive fully-connected layers. It is important to note, that the final dense layer of the classifier is not replaced. The modification is shown in Figure 33

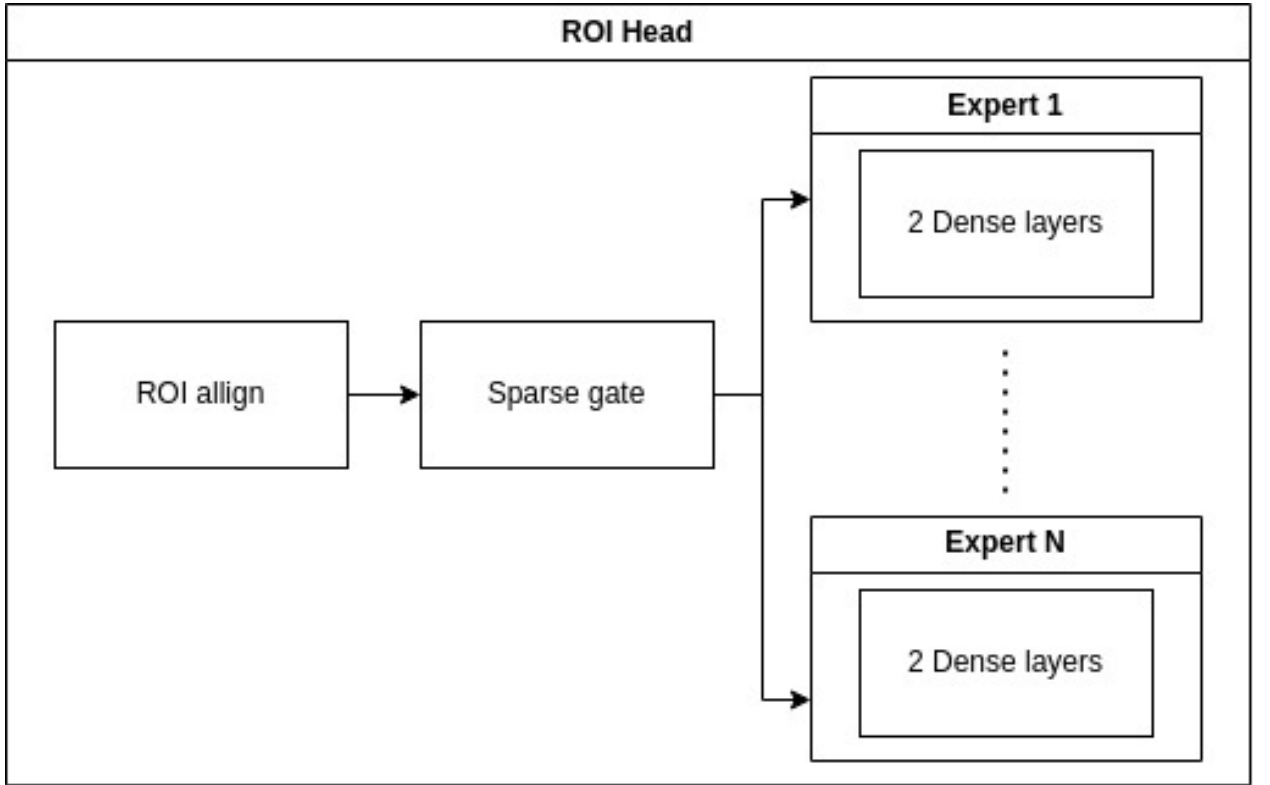


Figure 33 — Switch transformer inside ROI head of Faster RCNN.

Table 4 — class-wise mAP of Swin-based Faster RCNN, and the proposed model, when trained on both datasets together.

model	crazing	inclusion	patches	pitted surface	rolled in scale	scratches	mAP
swin-t	0.38	0.84	0.9	0.77	0.5	0.8	0.7
swin-s	0.41	0.8	0.86	0.73	0.44	0.79	0.67
swin t + s	0.33	0.81	0.91	0.77	0.5	0.8	0.69
Switch in fc	0.4	0.77	0.9	0.78	0.45	0.81	0.69

Training on both NEU-DET and oil datasets

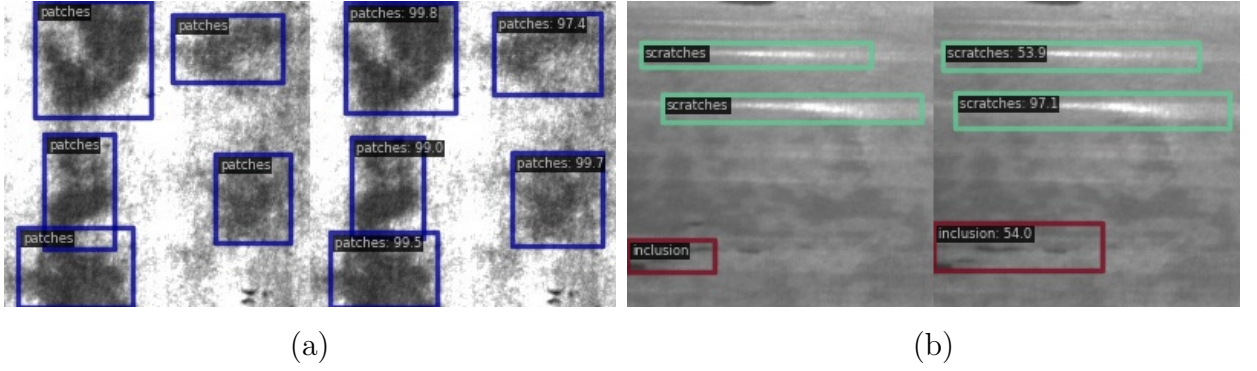


Figure 34 — Correct predictions of Swin-based Faster RCNN

2.4 Robustness Analysis

Vision-based deep learning detectors have shown high potential for detecting industrial defects for inspection. However, when deploying detectors on a quadcopter, they fail to recognize defects due to the variability of inspection conditions, which we call modifiers. This section starts by reviewing the modifiers used in the experiments. Then, it provides a numerical analysis of the robustness of DKAN model, lastly it analyzes the performance of Faster RCNN, which is based on Swin-t transformer as the backbone.

2.4.1 Modifiers

Three modifiers were used mainly in the experiments, which provides numerical analysis. The modifiers are: blur, light exposure, and affine transformation. In other experiments, which rely on visual analysis, additional modifiers were applied, which are: zoom blur, fog, contrast, and motion blur

Gaussian Blur

The blur applied in the experiments is the Gaussian blur following the formula below, even though zoom blur will be applied later.

$$G(x, y; \sigma) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}} \quad (16)$$

Affine transformation

The affine transformation modifier uses both rotation and shear angles to simulate the angle of the quadcopter. The formulas of rotations and shear

matrices are shown below:

$$R = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 1 \\ 0 & 0 & 1 \end{bmatrix} \quad (17)$$

$$S = \begin{bmatrix} 1 & k & 0 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix} \quad (18)$$

where k is the shear coefficient. Rotating the image synthetically causes loss of information of the image, as parts of it go out of the borders. To reduce this loss, the rotation is applied only around the center of the image. This requires translate matrices to be applied; one translation to move the coordinates to the center of the image, and another one to move them back to the origin.

$$L_{pre} = \begin{bmatrix} 1 & 0 & l_x \\ 0 & 1 & l_y \\ 0 & 0 & 1 \end{bmatrix} \quad (19)$$

$$L_{post} = \begin{bmatrix} 1 & 0 & -l_x \\ 0 & 1 & -l_y \\ 0 & 0 & 1 \end{bmatrix} \quad (20)$$

The final transformation is shown below:

$$T = L_{pre} S R L_{post} \quad (21)$$

Light exposure

The transformation is applied by converting the color space into HSV space. Then applying multiplicative random noise to the value axis, and then convert it back to the rgb space.

2.4.2 Robustness analysis under few-shot setting

This part investigates the affect of modifiers on the performance of Faster RCNN, FSCE, and DKAN, which were trained under few-shot setting.

It starts by conducting robustness analysis of the three models for affine transformations only. Then the model with the best performance is chosen to be analyzed on the other two modifiers when applied during testing, or during training and testing together.

At first, affine transformation is applied by randomly choosing an angle between 15 - 30 degrees, which a fixes shear angle of 15 degrees.

The results show that the models Faster RCNN, DKAN and FSCE are not robust to inspection conditions and cannot be relied on for inspection from a quadcopter.

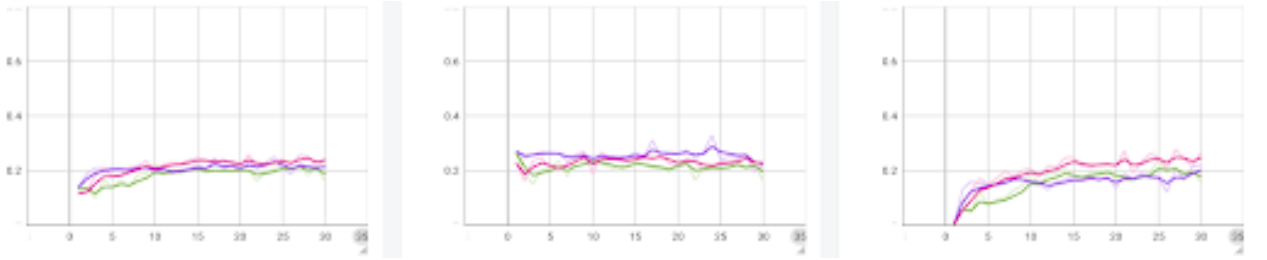


Figure 35 — Comparing mAP of Faster RCNN, FSCE, and DKAN on test data with affine transformation applied. A) Results on all classes. B) Results on base classes. C) Results on novel classes. Purple, pink, and green indicate DKAN, FSCE, and Faster RCNN respectively.

It was observed that FSCE shows the highest mAP for novel classes, but its performance on base classes is close to the performance of the re-trained Faster RCNN. DKAN shows less drop for base classes, but average performance on novel classes. Numerical results are shown in Table 5.

Table 5 — mAP of Faster RCNN, FSCE, and DKAN with and without applying modifiers. The last three columns indicate the results on base classes, novel classes, and all classes together.

Model	Modifiers	mAP Base	mAP Novel	mAP ALL
Faster RCNN	No	0.55	0.38	0.45
	Yes	0.21	0.25	0.23
FSCE	No	0.57	0.35	0.46
	Yes	0.22	0.3	0.26
DKAN	No	0.56	0.35	0.45
	Yes	0.28	0.22	0.25

While the performance of the models is acceptable in the nominal setting, it suffered from a sharp drop after modifiers were applied. Based on this experiment, the DKAN model showed the highest reserve of performance on base classes, and therefore was used in the following experiments. To investigate the robustness of the model for modifiers, blur, light exposure, and rotations were applied with three different difficulty levels. Easy, medium, and hard levels correspond to ranges of [15-30], [30-60], [60-90] for rotations, [10-15], [15-20], [20-25] for blur, and [0-0.2], [0.2-0.4], [0.4-0.6] for exposure. Exposure was applied in both positive and negative values. The results of the experiment are depicted in table 6.

Table 6 — mAP of DKAN on different levels of difficulties of the modifiers.

Modifier	Easy (30%)	Medium (60%)	Hard (90%)
Blur	0.62	0.44	0.33
Rotations	0.25	0.17	0.14
Exposure	0.64	0.57	0.50
All modifiers	0.22	0.09	0.04

A large drop in the performance was observed when rotations were applied. The easiest level of rotations, representing a range between 15 and 30 degrees, shows unreliable performance. The model shows better robustness to the other 2 modifiers, with exposure having less influence than blur. The mAP drops further as the difficulty of the modifier increases. When applying all modifiers together, the mAP gets to zero as the difficulty reaches its highest level. As an attempt to improve the performance of the model, another experiment is conducted, where DKAN is trained on the easy level of the modifiers. The results are shown in Table 7.

The results showed high robustness to blur with all different difficulties. When exposure was applied, the model showed a slightly lower performance, while the drop is higher for rotations. The zero-convergence observed earlier when applying all modifiers together was canceled when the model was trained on modified data. All previous experiments showed that FSCE and DKAN suffer from low robustness to quadcopter conditions for inspection. Therefore, they cannot be deployed for defect detection. Training these models on modified data does not improve the performance enough to rely on it.

Table 7 — mAP of DKAN on different levels of difficulties of the modifiers, after the easy level of modifiers was applied during training.

Modifier	Easy (30%)	Medium (60%)	Hard (90%)
Blur	0.58	0.61	0.60
Rotations	0.24	0.16	0.15
Exposure	0.54	0.53	0.47
All modifiers	0.26	0.20	0.15

To improve the robustness of object detectors, other methods which do not use CNNs as a backbone were considered.

2.4.3 Analyzing the Swin-based FASTER RCNN

In this part, the robustness analysis is conducted using the developed GUI. It does not produce numerical results, but rather follows the visual observation of the results. The reason for not using numerical calculations, is because images have different degrees of brightness and darkness (these modifiers are inherent in the dataset). What we are interested in is to observe the behavior of the model when the object is still seen, which requires specific values of the modifiers for every image individually. In this section, the modifiers used are: affine transformation, motion blur, light exposure, contrast, and zoom blur.

For defects with gaps between their features, like crazing, pitted-surface, and rolled-in scale (from now called group 1 defects), the model showed the worse robustness among other defects, and did not show robustness to blur. The reason of that is that these defects consist of small features combined in a bigger bounding box. When a modifier is applied, these small features disappear, making it impossible to detect by human eye, or by a model. For other defects, it takes more intensive modifiers to make defects disappear. Examples of defect disappearance is shown in Figure 36. Another issue found with intensive modifiers is misclassification. Misclassification happen when defects with distinct features disappear partially, or are collapsed into smaller portions with gaps between them, transforming themselves into defect with similar features to group 1 defects.

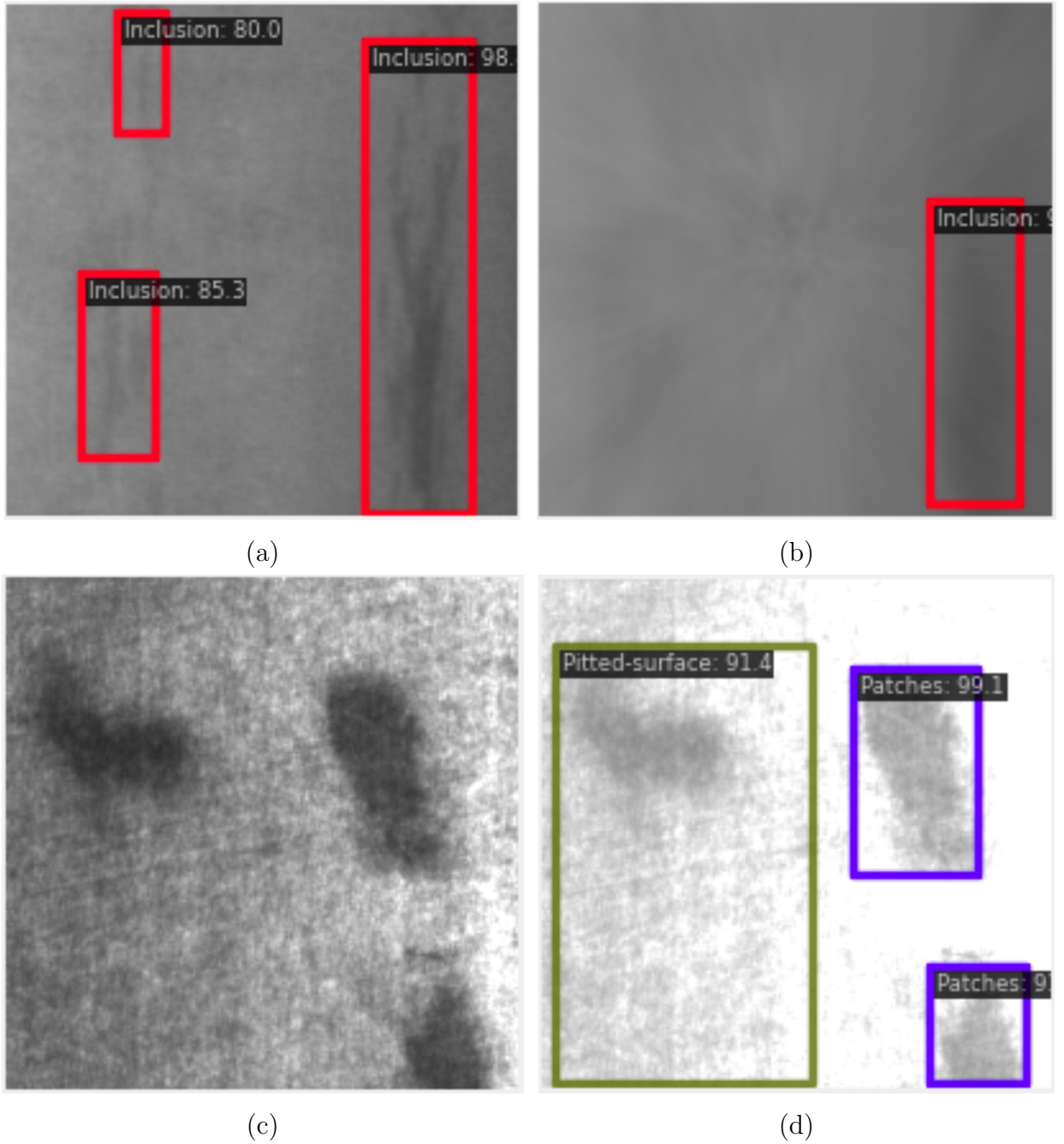


Figure 36 — Problems faced with intensive modifiers.

Generally speaking, the model showed high robustness to affine transformations, fogs, motion blur, zoom blur, light exposure, and contrast

With intensities that are challenging to detect with bare eye, the model could successfully detect the defects in many scenarios. Examples of that are shown in Figures 37, and 38

Remarkably, Swin-based Faster RCNN showed robustness to affine transformations, while on the contrary, the performance of CNN-based Faster RCNN collapsed with this transformation was applied.

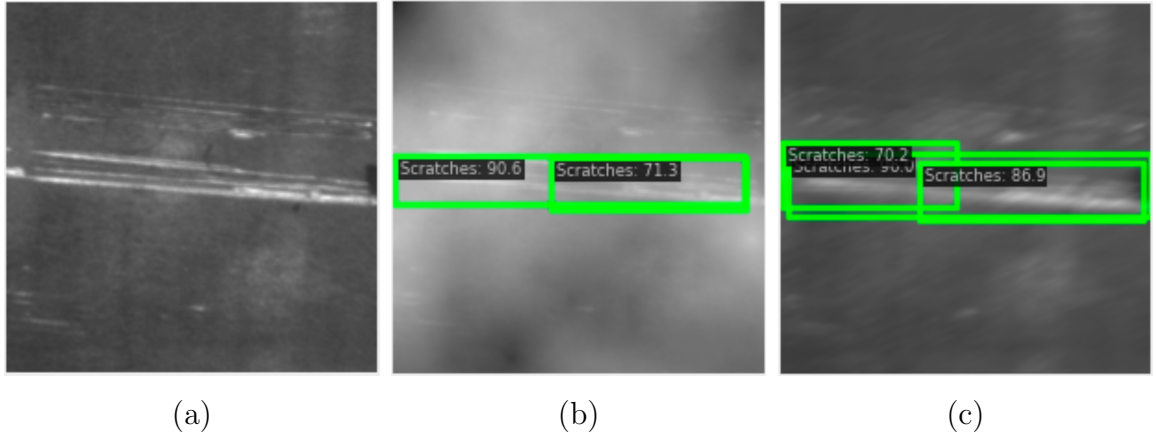


Figure 37 — (a) Original image. (b) The image after synthetically adding fog. (c) The image after motion blur is applied.

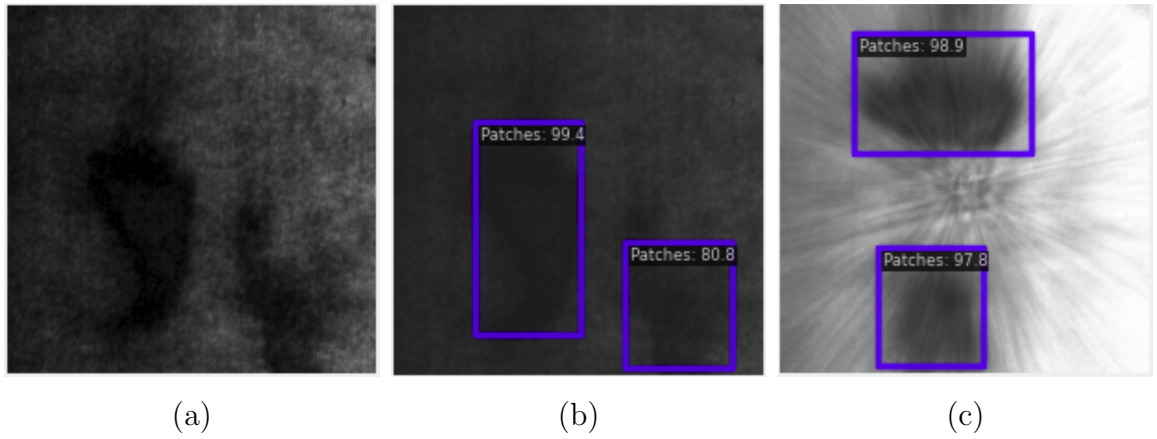


Figure 38 — (a) Original image. (b) The image with contrast. (c) The image with zoom blur.

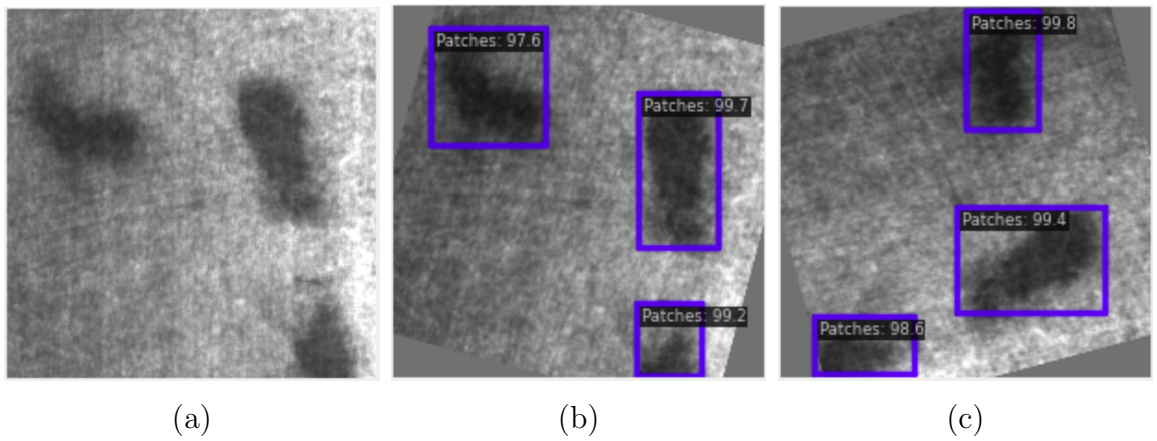


Figure 39 — (a) Original image. (b) The image with rotation of 15 degrees. (c) The image after rotation of 75 degrees.

2.5 Data cleaning

Data cleaning is a crucial step to let the performance coincide with the metrics used for measuring it.

What is interesting about the results, is that the swin model produces its correct predictions with a confidence close to 100% with high precision. However, the results show mAP over 90 for only one class. The reason for that has to do with the dataset itself. NEU-DET dataset has correct labeling of the training data, but it lacks quality labeling in the testing data. The labeling errors include overlooked, badly located objects. When the model detects the unlabeled defects, the mAP drops due to an additional false positive.

The misleading labels emerging from overlooked labels are shown in Figure 40. To deal with this issue, data cleaning was done by removing images with such issues. As a result, 11 images were classified as dirty images, and consequently they were removed.

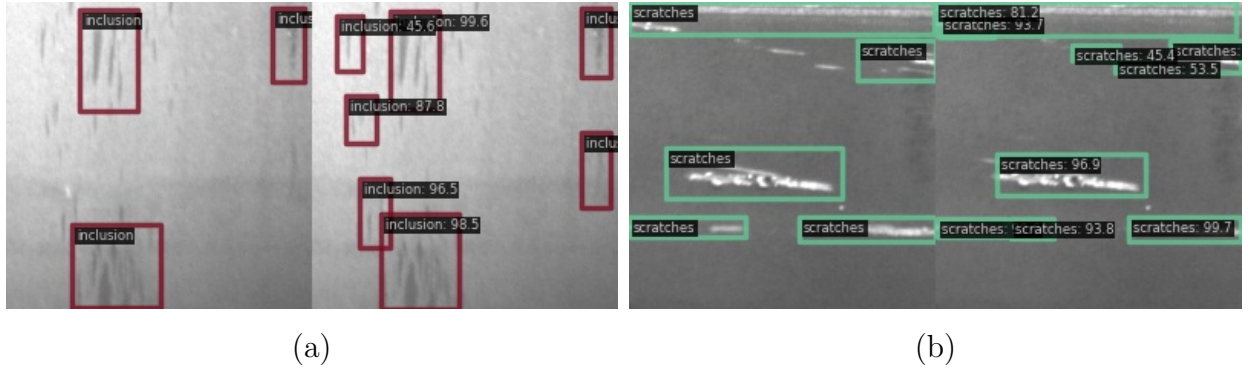


Figure 40 — The problem of missing labels.

Another issue, which contributes to the misleading drop in mAP, is the divided labels. When there are defects with non-rectangular shape, the model prefers to produce a big bounding box, which contains the whole defect instead of producing two bounding boxes to cover the shape. In the test dataset, such defects are labeled with two bounding boxes. During calculation of mAP, if the IOU of the predicted bounding box and each of the labeled boxes is smaller than 0.5, then the prediction is considered wrong. In this case, 2 false negative and 1 false positive will be counted, affecting both the recall and the precision. An illustration of this issue is demonstrated in Figure 41

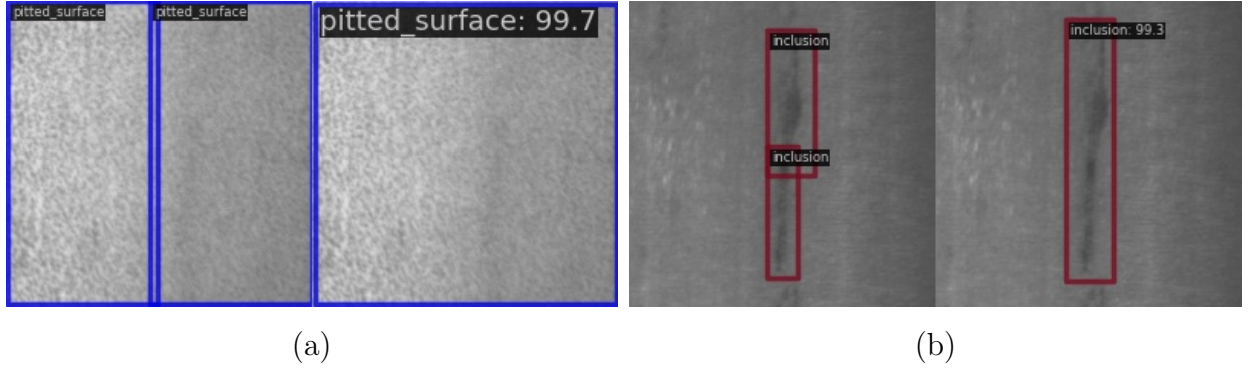


Figure 41 — (a) The problem of divided labels. (b) The problem of non-rectangular defects.

To deal with this issue, a new metric was proposed, where the detection task is converted into a segmentation task. Instead of matching bounding boxes predictions with labels, the predictions are converted into segmentation maps. Bounding boxes of the same class are united into one mask for labels, and one mask for predictions. Then IOU is calculated class-wise between predictions and labels, and the IOU threshold is applied to calculate precision and recall, and consequently mAP. Uniting bounding boxes of the same class is a necessary step, as the model produces less predictions than the number of labels. Using this metric, along with data cleaning raised the mAP to 0.85.

	crazing	inclusion	patches	pitted surface	rolled in scale	scratches
swin-t	0.75	0.72	0.93	0.98	0.74	0.96

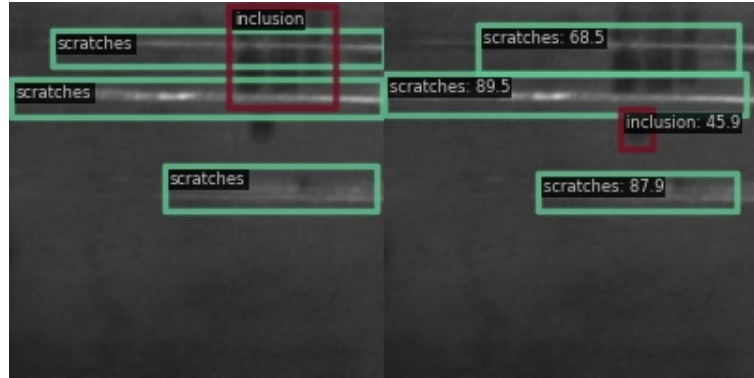
Table 8 — class-wise mAP of Swin-based Faster RCNN, and the proposed model, when trained on both datasets together.

Limitations

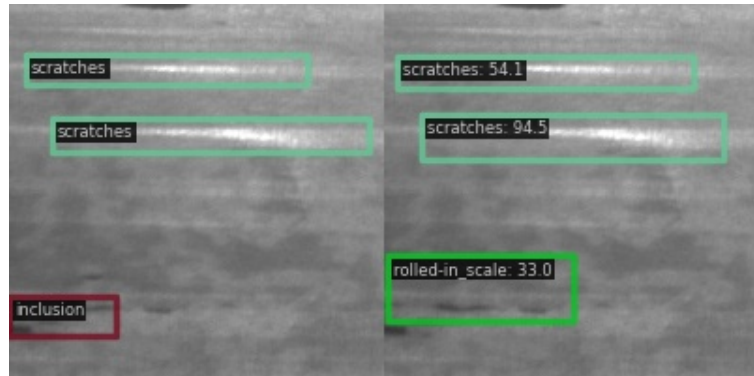
Despite the good performance of the model, there are still some drawbacks which are seen in specific scenarios. For examples, when defects of different classes occur above each other, the model fails to detect these classes. In other cases, misclassification was observed in a small portion. The following problems are shown below.



(a)



(b)



(c)

Figure 42 — Limitations of Swin-based Faster RCNN (a) and (c) misclassification. (b) overlapping defects of different classes.

2.6 Discussion

Optimizing Faster RCNN is not sensitive to different values of learning rate as was shown in the experiments, where there was no tangible difference in a range of learning rate values between 0.02 - 0.002. Retraining Faster RCNN triggers catastrophic forgetting, and is inefficient. Using 2-phase training pipeline improves the performance, especially when modifiers are applied, though the improvement does not exceed 10% in mAP. Despite the small improvement in mAP, DKAN showed less misclassification errors compared to FSCE. After analyzing DKAN on different modifiers, it was shown that DKAN has high robustness against blur and light exposure, and this robustness is magnified when the model is trained on the modifiers themselves. Numerically, model's mAP changed by no more than 4% for blur, and no more than 10% for light exposure.

Resnet backbone has high sensitivity for even small rotations as 15 degrees. It can't give reliable detections when the object is rotation, even with the absence of shear angle. This made the move to transformers more reasonable, where Swin-t transformer showed impressive robustness for different modifiers, compared to Resnet. Without considering modifiers, just replacing Resnet with Swin-t raised the performance from 0.65 on 3 classes into 0.7 on 6 classes. This performance surpassed the performance of YOLOv8 and DINO on both NEU-DET and oil spills datasets by at least 10%.

Blending MOE in the last layer of Swin-t gave similar results to not blending it, except that it overfitted some classes, and the performance on others dropped. Swin-s backbone showed lower mAP than swin-t, but 3% higher mAP for Crazing. After combining two models, one with swin-t and one with swin-s, the performance dropped, and it continued to drop when using MOE in the shared layers of the ROI head. As a result of all experiments, Faster RCNN with swin-t backbone outperformed all tested models in terms of mAP and in terms of robustness

CONCLUSION

This thesis provided solutions for 3 problems associated with visual anomaly detection: data scarcity, emergence of new data, and environmental conditions issues. Three models were tested to monitor the performance under few-shot setting, with an expanding classifier's head to simulate the emergence of novel classes. The results show that applying knowledge distillation is an effective technique to reduce the drop in the performance when training the model on novel types of defects. Resnet backbone for Faster RCNN was proven to be weak against rotated objects, though it showed robustness for blur and brightness. Transformer-based backbones showed higher performance compared to Resnet, where they showed high robustness for half of the classes for all applied modifiers, and acceptable robustness for modifiers with lower severity for the other half of the classes. Faster RCNN with swin-t backbone outperformed models, which are using MOE in the backbone, or in the ROI head, as well as two combined swin-based models. Data cleaning was shown to be an essential step to apply before testing, as applying it improved the performance from 0.7 mAP to 0.85 on NEU-DET dataset. This model is ready to be launched on a quadcopter. A control system can be designed and implemented to perform inspection and give command to the quadcopter to avoid light exposure or move away from shadow. A system for planning the trajectory of the quadcopter can be designed. The model implemented in this thesis forms the base for future work to maintain accurate and reliable inspection.

BIBLIOGRAPHY

1. *Wan Q., Gao L., Li X., Wen L.* Industrial image anomaly localization based on Gaussian clustering of pretrained feature // IEEE Transactions on Industrial Electronics. — 2021. — Vol. 69, no. 6. — P. 6182–6192.
2. *Jiao Z., Jia G., Cai Y.* A new approach to oil spill detection that combines deep learning with unmanned aerial vehicles // Computers & Industrial Engineering. — 2019. — Vol. 135. — P. 1300–1311.
3. *Jing S., Yujun T., Lili R.* Frequency and Causes of Oil Spill Accidents from Ships and Storage Tanks in Quanzhou, China // Journal of Resources and Ecology. — 2023. — Vol. 14, no. 2. — P. 391–398.
4. *Sun B., Li B., Cai S., Yuan Y., Zhang C.* Fsce: Few-shot object detection via contrastive proposal encoding // Proceedings of the IEEE/CVF conference on computer vision and pattern recognition. — 2021. — P. 7352–7362.
5. *Chen L., Yu C., Chen L.* A new knowledge distillation for incremental object detection // 2019 International Joint Conference on Neural Networks (IJCNN). — IEEE. 2019. — P. 1–7.
6. *Ren S., He K., Girshick R., Sun J.* Faster R-CNN: Towards real-time object detection with region proposal networks // IEEE transactions on pattern analysis and machine intelligence. — 2016. — Vol. 39, no. 6. — P. 1137–1149.
7. *Hinton G., Vinyals O., Dean J.* Distilling the knowledge in a neural network // arXiv preprint arXiv:1503.02531. — 2015.
8. *Sun C., Gao L., Li X., Gao Y.* A new knowledge distillation network for incremental few-shot surface defect detection // arXiv preprint arXiv:2209.00519. — 2022.
9. *Vaswani A., Shazeer N., Parmar N., [et al.].* Attention is all you need // Advances in neural information processing systems. — 2017. — Vol. 30.
10. *Dosovitskiy A., Beyer L., Kolesnikov A., [et al.].* An image is worth 16x16 words: Transformers for image recognition at scale // arXiv preprint arXiv:2010.11929. — 2020.

11. *Liu Z., Lin Y., Cao Y., [et al.]*. Swin transformer: Hierarchical vision transformer using shifted windows. — 2021.
12. *Shazeer N., Mirhoseini A., Maziarz K., [et al.]*. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer // arXiv preprint arXiv:1701.06538. — 2017.
13. *Yu Q., Kavitha M.S., Kurita T.* Mixture of experts with convolutional and variational autoencoders for anomaly detection // Applied Intelligence. — 2021. — Vol. 51, no. 6. — P. 3241–3254.
14. *Jain Y., Behl H., Kira Z., Vineet V.* DAMEX: Dataset-aware Mixture-of-Experts for visual understanding of mixture-of-datasets // Advances in Neural Information Processing Systems. — 2024. — Vol. 36.
15. *Li B., Shen Y., Yang J., [et al.]*. Sparse mixture-of-experts are domain generalizable learners // arXiv preprint arXiv:2206.04046. — 2022.
16. *Riquelme C., Puigcerver J., Mustafa B., [et al.]*. Scaling vision with sparse mixture of experts // Advances in Neural Information Processing Systems. — 2021. — Vol. 34. — P. 8583–8595.
17. *Fedus W., Zoph B., Shazeer N.* Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity // Journal of Machine Learning Research. — 2022. — Vol. 23, no. 120. — P. 1–39.
18. *Xue F., He X., Ren X., Lou Y., You Y.* One student knows all experts know: From sparse to dense // arXiv preprint arXiv:2201.10890. — 2022.
19. *Ridzuan F., Zainon W.M.N.W.* A review on data cleansing methods for big data // Procedia Computer Science. — 2019. — Vol. 161. — P. 731–738.
20. *Tkachenko U., Thyagarajan A., Mueller J.* Objectlab: Automated diagnosis of mislabeled images in object detection data // arXiv preprint arXiv:2309.00832. — 2023.
21. *Al-Battbootti M.J.H., Nicolae G., MARIN I.* Detection and Analysis of Oil Spill using Image Processing // International Journal of Advanced Computer Science and Applications. — 2022. — Vol. 13, no. 4.

22. *Chang S.E., Stone J., Demes K., Piscitelli M.* Consequences of oil spills: a review and framework for informing planning // *Ecology and Society*. — 2014. — Vol. 19, no. 2.
23. *Caron M., Touvron H., Misra I., [et al.]*. Emerging properties in self-supervised vision transformers // *Proceedings of the IEEE/CVF international conference on computer vision*. — 2021. — P. 9650–9660.

APPENDIX

Predictions of Faster RCNN with the Swin transformer in the backbone.

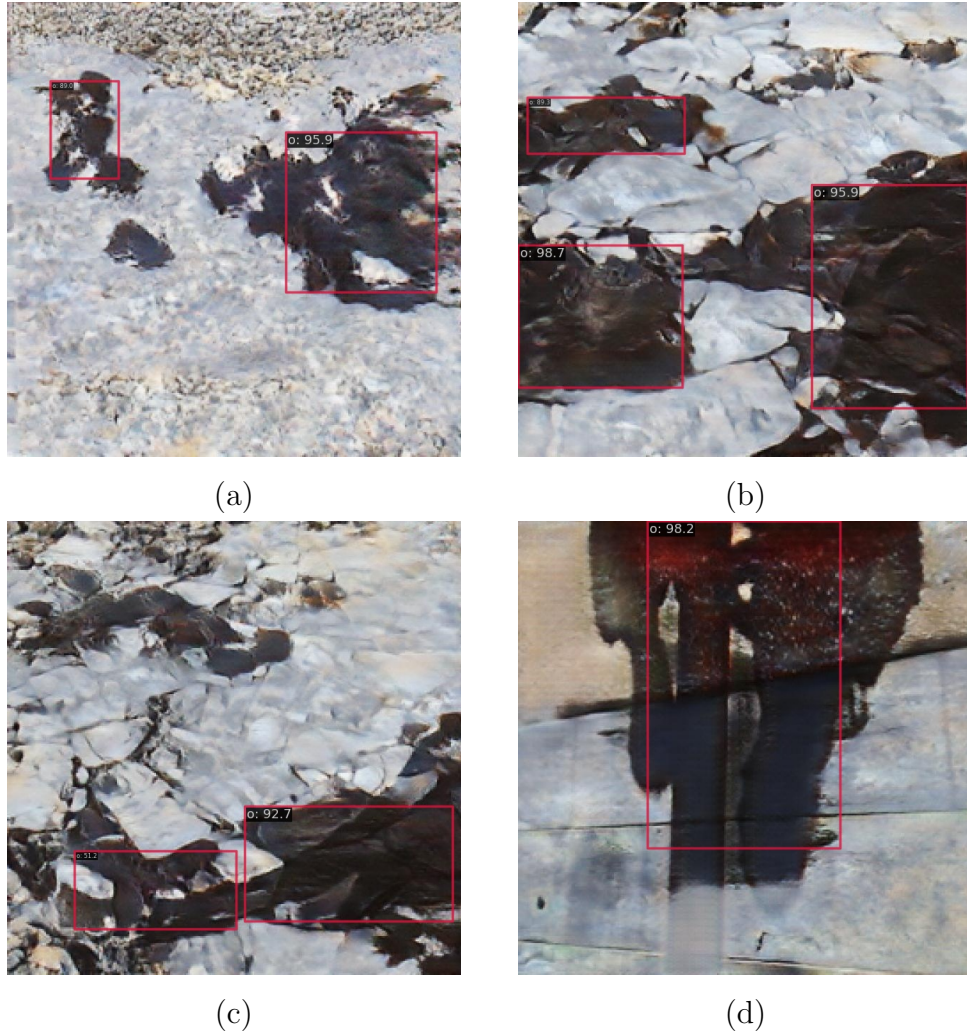
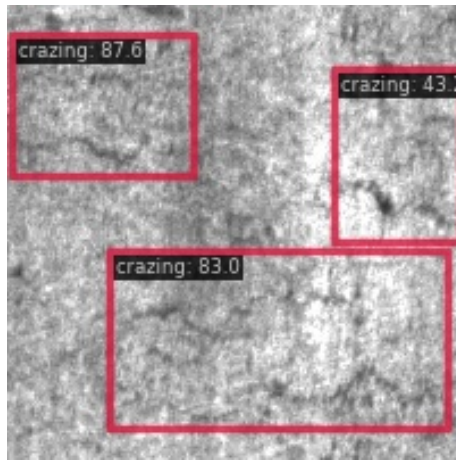
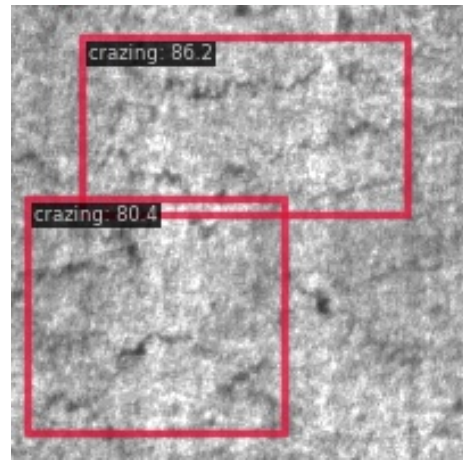


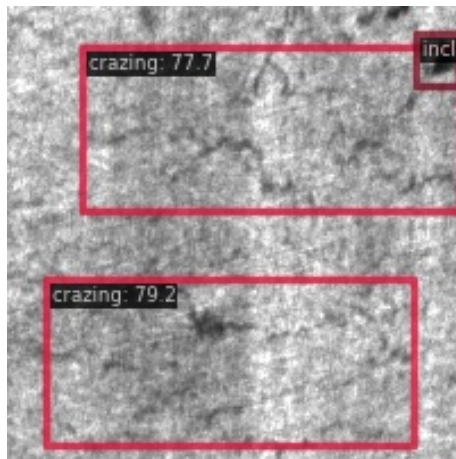
Figure 43 — samples of predictions of oil



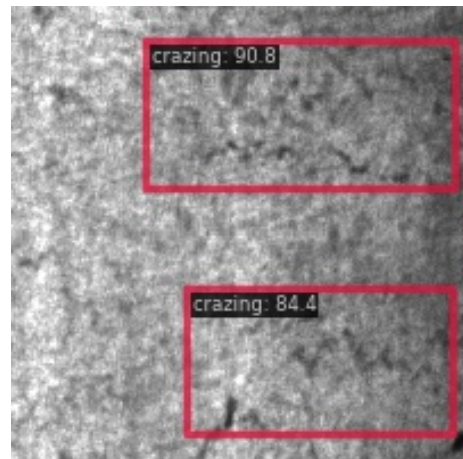
(a)



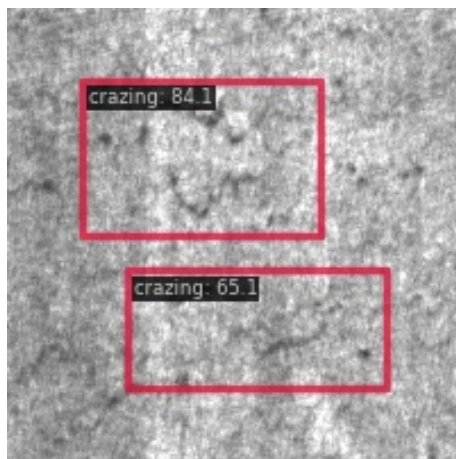
(b)



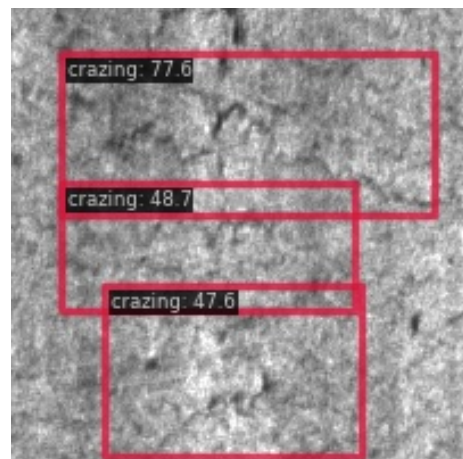
(c)



(d)

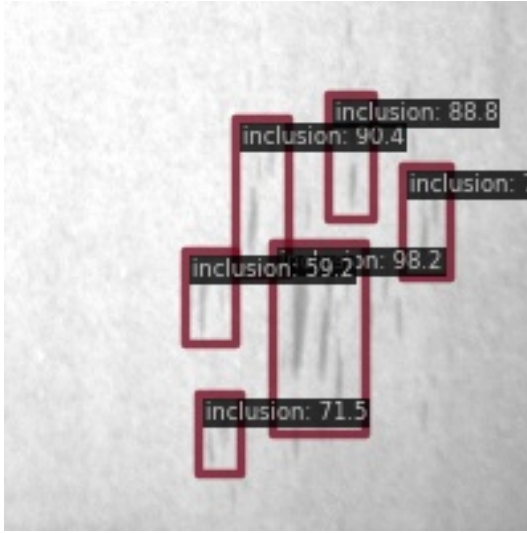


(e)

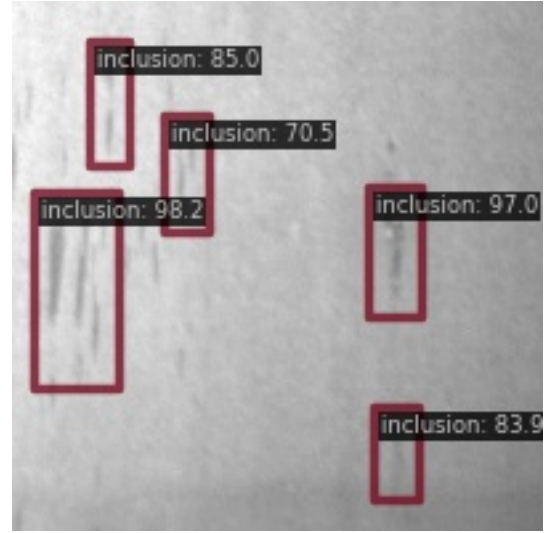


(f)

Figure 44 — samples of predictions of crazing



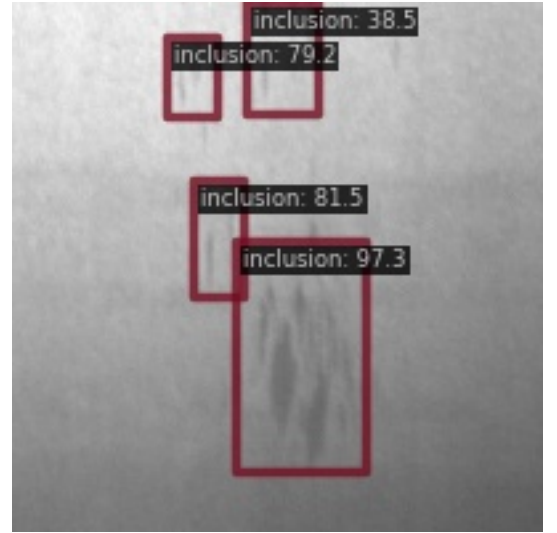
(a)



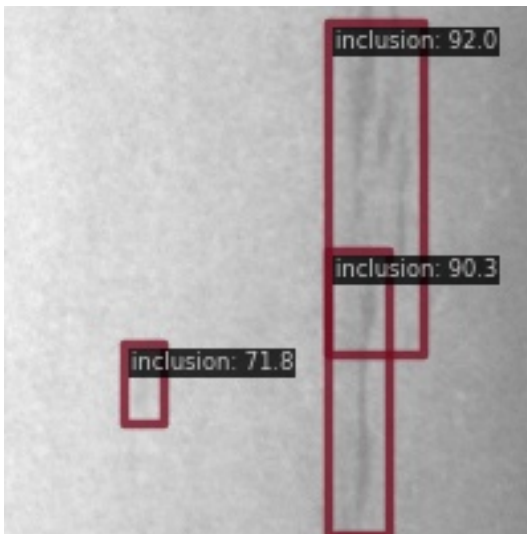
(b)



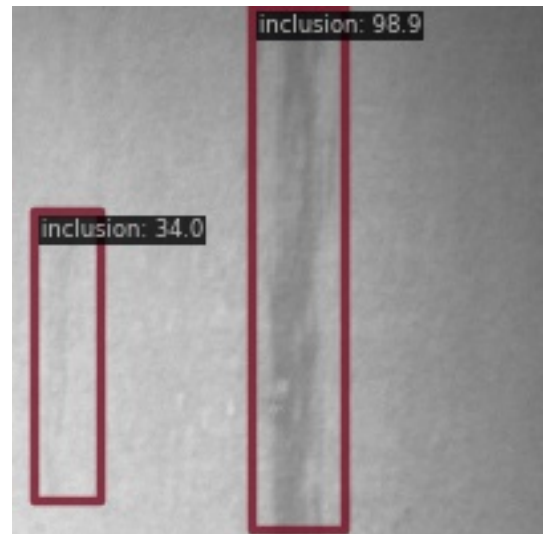
(c)



(d)

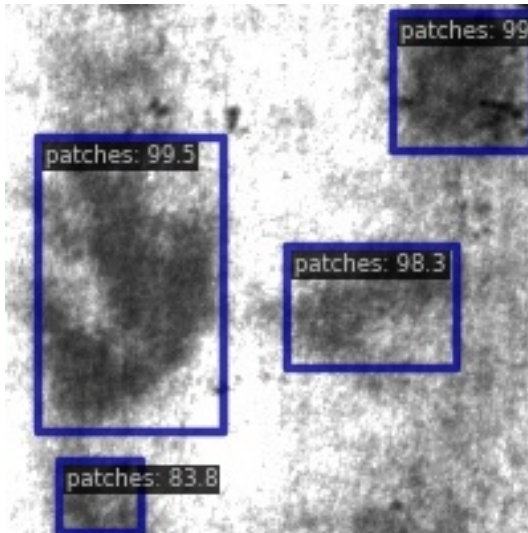


(e)

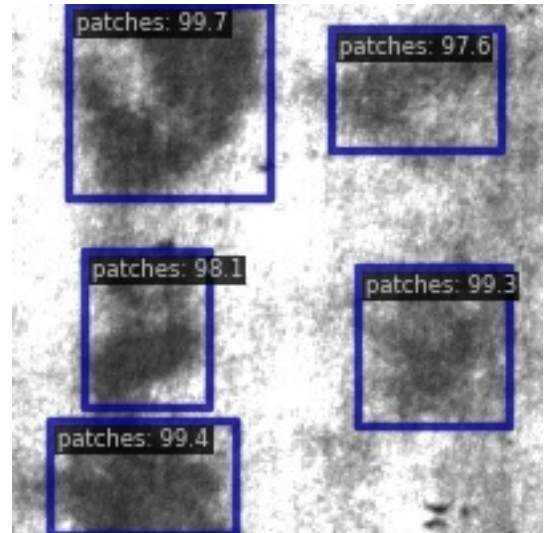


(f)

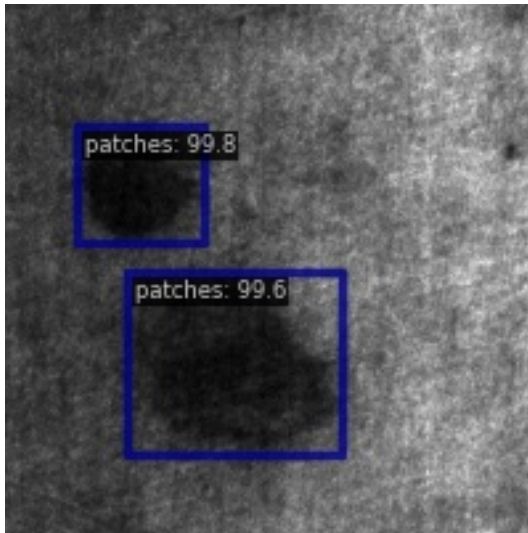
Figure 45 — samples of predictions of inclusion



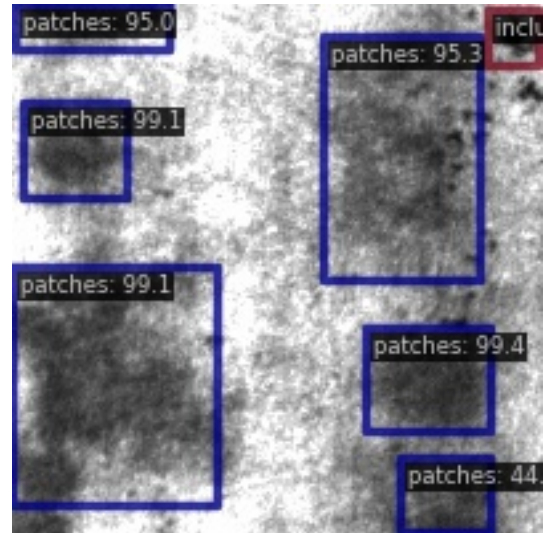
(a)



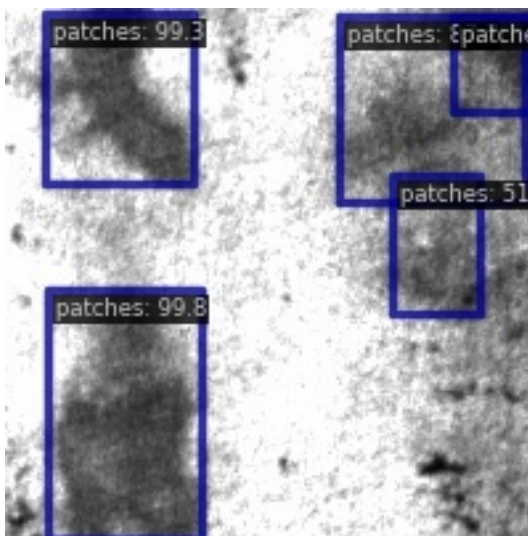
(b)



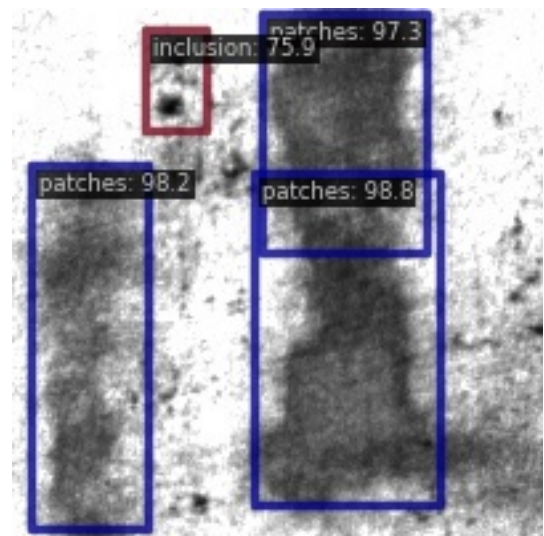
(c)



(d)

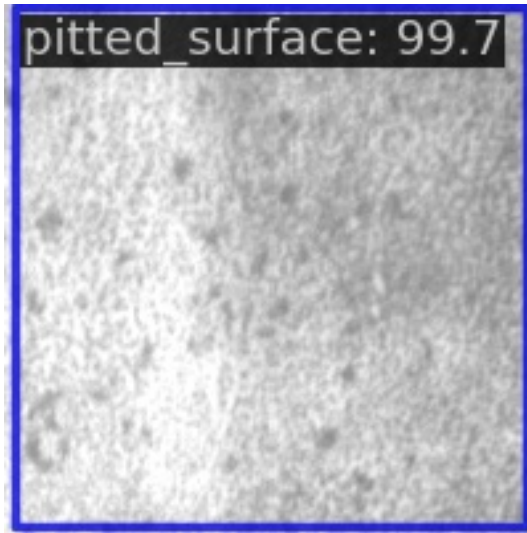


(e)

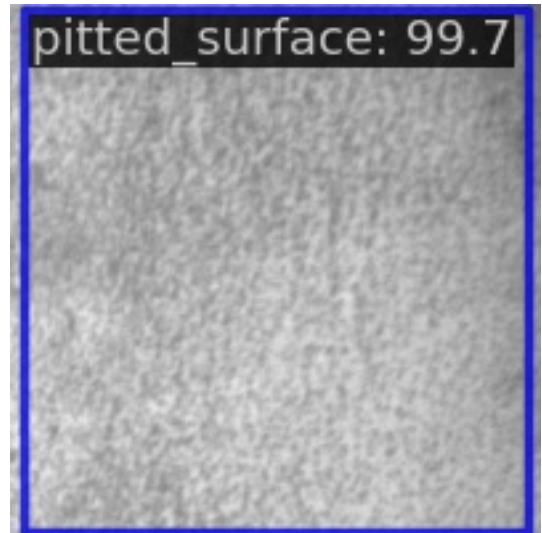


(f)

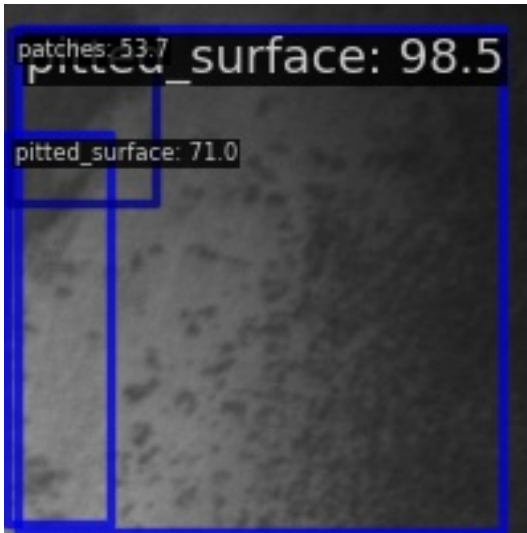
Figure 46 — samples of predictions of patches



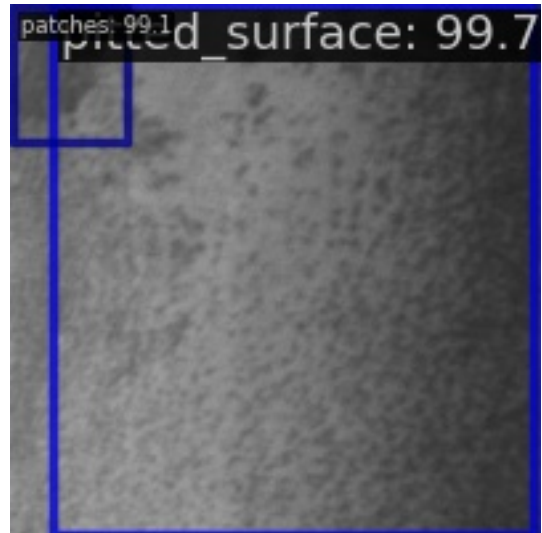
(a)



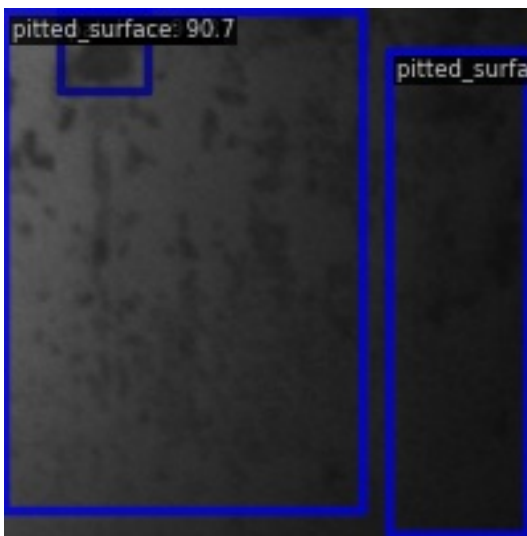
(b)



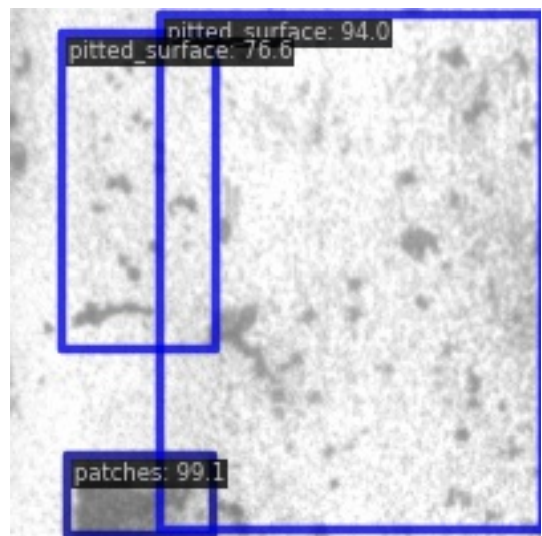
(c)



(d)

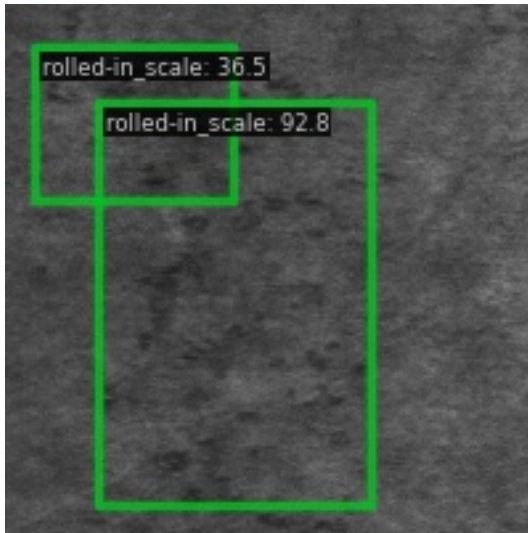


(e)

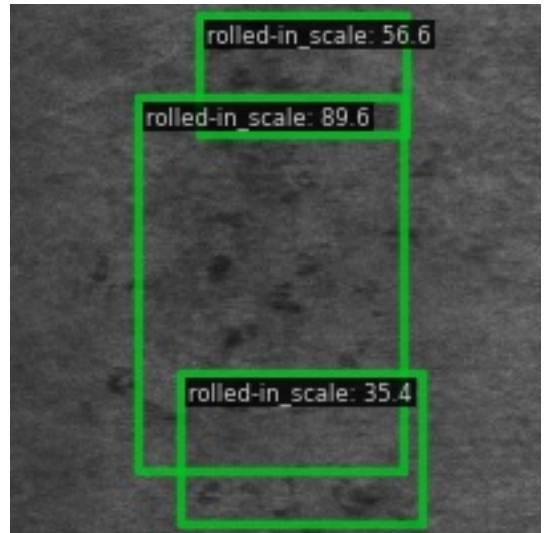


(f)

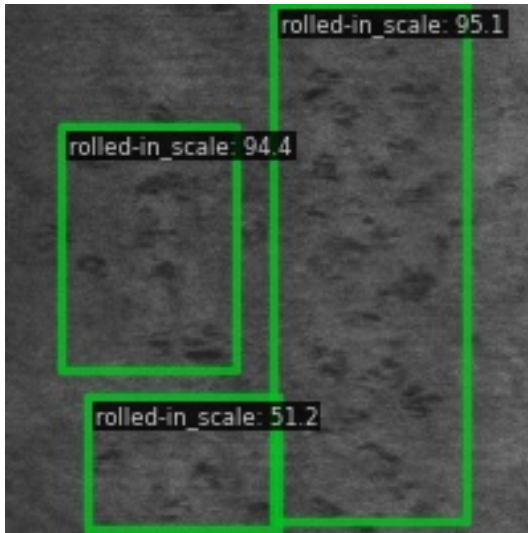
Figure 47 — samples of predictions of pittedsurface



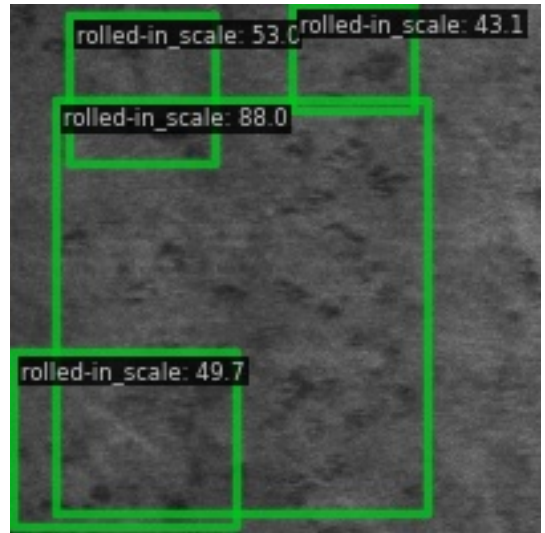
(a)



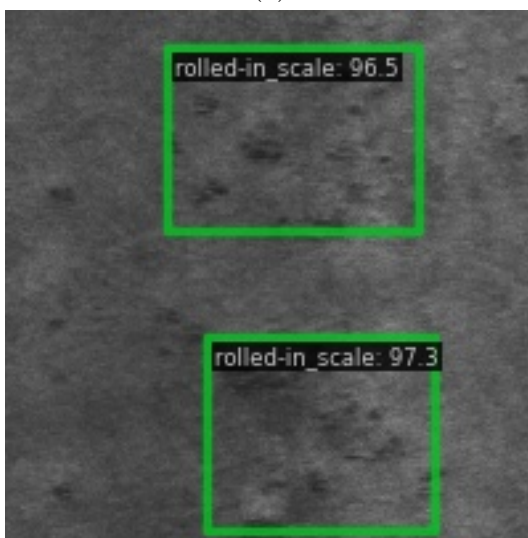
(b)



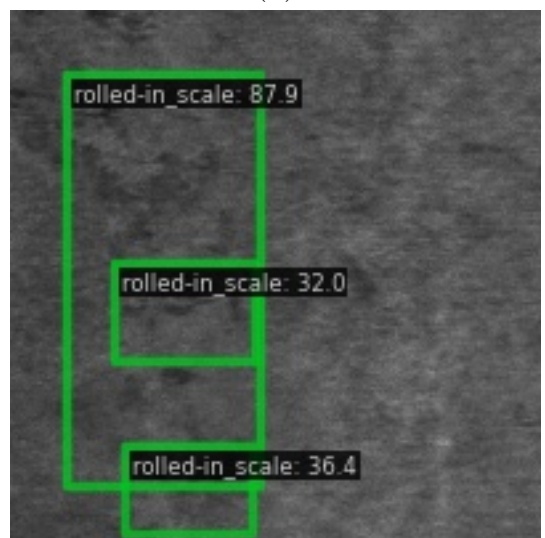
(c)



(d)

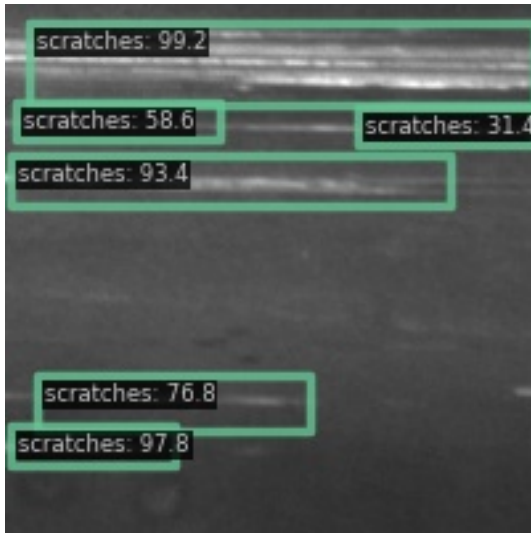


(e)



(f)

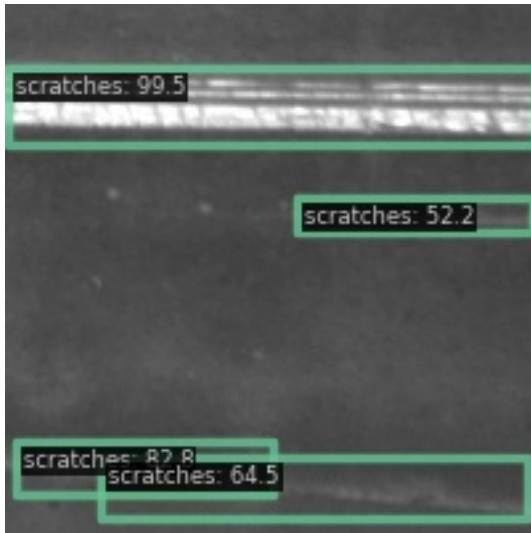
Figure 48 — samples of predictions of rolled-in scale



(a)



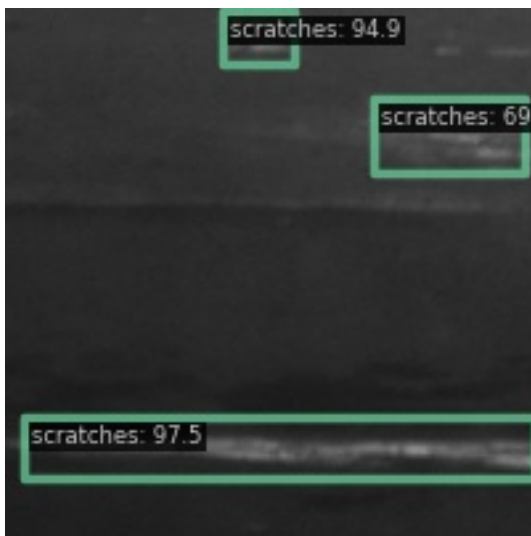
(b)



(c)



(d)



(e)



(f)

Figure 49 — samples of predictions of scratches